

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT HƯNG YÊN



GIÁO TRÌNH
PHÁT TRIỂN PHẦN MỀM HƯỚNG DỊCH VỤ

Tác giả: NGUYỄN HỮU ĐÔNG

Đơn vị công tác: Khoa Công nghệ thông tin

Hưng Yên 2026

LỜI GIỚI THIỆU

Tài liệu được biên soạn nhằm cung cấp một lộ trình học tập có hệ thống về phát triển **Lập trình hướng dịch vụ**. Nội dung kết hợp kiến thức nền tảng, ví dụ mã nguồn, bài tập thực hành và một đồ án tổng hợp, giúp học viên tiếp cận đầy đủ vòng đời xây dựng một dịch vụ backend hiện đại.

Giáo trình ưu tiên phương pháp “học đi đôi với làm”: mỗi bài đều xác định mục tiêu, trình bày kiến thức cốt lõi, minh họa bằng mã nguồn và kết thúc bằng câu hỏi hoặc nhiệm vụ thực hành. Người học nên thực hiện tuần tự các bài và duy trì một repository Git xuyên suốt khóa học.

Các ví dụ tập trung vào ASP.NET Core, SQL Server, ADO.NET, Dapper, JWT, Redis, Serilog, xUnit, Docker và GitHub Actions. Giảng viên có thể điều chỉnh thời lượng, phạm vi công nghệ và mức độ của đồ án tùy theo năng lực đầu vào của lớp.

THÔNG TIN CHUNG VỀ KHÓA HỌC

Tên khóa học	Lập trình Web API chuyên nghiệp với ASP.NET Core
Đối tượng	Sinh viên CNTT; lập trình viên có nền tảng C# và SQL cơ bản
Thời lượng đề xuất	60 tiết: 30 tiết lý thuyết, 30 tiết thực hành
Hình thức	Trực tiếp hoặc kết hợp trực tuyến; khoảng 70% thực hành
Công nghệ chính	ASP.NET Core, SQL Server, ADO.NET, Dapper, JWT, Redis, Serilog, xUnit, Docker, GitHub Actions
Sản phẩm đầu ra	Một hệ thống Web API quản lý bán hàng có xác thực, phân quyền, logging, caching, testing và triển khai container

MỤC TIÊU ĐẦU RA

- Thiết kế endpoint theo nguyên tắc REST, lựa chọn đúng HTTP method, status code và quy ước URI.
- Xây dựng Web API bằng Controller hoặc Minimal API; thực hiện routing, model binding, validation và tài liệu OpenAPI.

- Truy cập SQL Server an toàn bằng ADO.NET và Dapper; quản lý transaction và tổ chức Repository Pattern.
- Triển khai JWT Access Token, Refresh Token, phân quyền theo vai trò/chính sách và các biện pháp bảo vệ API.
- Xử lý lỗi toàn cục, structured logging, caching, rate limiting, versioning và health checks.
- Tổ chức mã nguồn theo Clean Architecture; hiểu ranh giới giữa monolith và microservices.
- Viết unit test, integration test; đóng gói Docker và thiết lập pipeline CI/CD cơ bản.
- Phân tích, xây dựng, trình diễn và bảo vệ một dự án Web API hoàn chỉnh.

ĐIỀU KIỆN TIỀN QUYẾT

- C# cơ bản: lớp, interface, generic, async/await, LINQ và xử lý ngoại lệ.
- SQL cơ bản: SELECT, JOIN, INSERT, UPDATE, DELETE, khóa chính/khóa ngoại.
- Biết sử dụng Git ở mức clone, commit, branch và push.
- Máy tính cài .NET SDK, Visual Studio/VS Code, SQL Server, Postman/Bruno và Docker Desktop.

KẾ HOẠCH GIẢNG DẠY ĐỀ XUẤT

Bài	Chủ đề	Lý thuyết	Thực hành	Sản phẩm
1	Web API, REST, ASP.NET Core	3	3	API mẫu và middleware
2	Endpoint, binding, validation, Swagger, Postman	2	5	CRUD API không CSDL
3	ADO.NET, Dapper, transaction, repository	3	7	CRUD API kết nối SQL Server
4	Authentication, authorization, security	3	5	Module đăng nhập và phân quyền
5	Exception, logging, cache, performance	2	4	API có logging và Redis cache

Bài	Chủ đề	Lý thuyết	Thực hành	Sản phẩm
6	Clean Architecture, microservices, Docker	3	5	Solution đa tầng chạy Docker Compose
7	Unit test, integration test, CI/CD	2	5	Bộ test và workflow CI
8	Đồ án tổng hợp	0	8	Sản phẩm và báo cáo bảo vệ

PHƯƠNG PHÁP ĐÁNH GIÁ

Thành phần	Tỷ trọng	Yêu cầu	Minh chứng
Chuyên cần và bài tập	20%	Tham gia tối thiểu 80%; hoàn thành bài tập từng bài	Repository, commit, demo
Kiểm tra giữa khóa	30%	Xây dựng CRUD API kết nối CSDL, có JWT và validation	Mã nguồn và kiểm thử Postman
Bài tập lớn	50%	Hoàn thiện hệ thống, test, Docker, báo cáo và bảo vệ	Sản phẩm chạy được, tài liệu, thuyết trình

MỤC LỤC

LỜI GIỚI THIỆU	2
THÔNG TIN CHUNG VỀ KHÓA HỌC	2
MỤC TIÊU ĐẦU RA.....	2
ĐIỀU KIỆN TIỀN QUYẾT	3
KẾ HOẠCH GIẢNG DẠY ĐỀ XUẤT	3
PHƯƠNG PHÁP ĐÁNH GIÁ	4
BÀI 1: GIỚI THIỆU VỀ WEB API VÀ ASP.NET CORE.....	16
1.1. KHÁI NIỆM WEB API, RESTful API VÀ HTTP METHODS.....	16
1.1.1. Web API là gì?	16
1.1.2. Kiến trúc RESTful API	16
1.1.3. Các HTTP Methods phổ biến.....	16
1.2. BEST PRACTICES TRONG THIẾT KẾ RESTful API	17
1.3. CÁC CHUẨN GIAO TIẾP: REST vs GRAPHQL vs gRPC.....	18
1.4. CÀI ĐẶT MÔI TRƯỜNG PHÁT TRIỂN	18
1.5. TỔNG QUAN KIẾN TRÚC ASP.NET CORE	19
1.5.1. Cấu trúc thư mục tiêu chuẩn	19
1.5.2. File Program.cs - Trái tim của ứng dụng	19
1.6. MIDDLEWARE VÀ PIPELINE XỬ LÝ REQUEST	20
1.6.1. Middleware là gì?.....	20
1.6.2. Hướng dẫn tự tạo Custom Middleware.....	20
1.7. DEPENDENCY INJECTION (DI) CƠ BẢN	21
1.7.1. Khái niệm IoC và Dependency Injection.....	21

1.7.2. Vòng đời của Service (Service Lifetimes) trong ASP.NET Core.....	22
1.7.3. Minh họa code thực hành Dependency Injection.....	22
CÂU HỎI VÀ BÀI TẬP Củng Cố Bài 1	23
□ Câu hỏi lý thuyết:	23
□ Bài tập thực hành:.....	24
BÀI 2: XÂY DỰNG WEB API CƠ BẢN VÀ XỬ LÝ REQUEST	25
2.1. CONTROLLER-BASED API VÀ MINIMAL APIs.....	25
2.1.1. Controller-based API	25
2.1.2. Minimal APIs	25
2.2. ROUTING VÀ DATA BINDING	26
2.2.1. Attribute Routing	26
2.2.2. Data Binding Attributes	26
2.3. TRẢ VỀ JSON VÀ CẤU HÌNH SERIALIZATION.....	27
2.3.1. Định dạng kết quả trả về với IActionResult.....	27
2.3.2. Cấu hình JSON Serialization (System.Text.Json)	28
2.4. ĐẢM BẢO VỆ TOÀN DỮ LIỆU (DATA VALIDATION)	28
2.4.1. Sử dụng Data Annotations	28
2.4.2. Validation nâng cao với FluentValidation	29
2.5. TÍCH HỢP SWAGGER / OPENAPI CHO TÀI LIỆU API	30
2.5.1. Cấu hình Swagger nâng cao trong Program.cs	30
2.5.2. Thêm Mô tả chi tiết cho Endpoint (XML Documentation)	31
2.6. KIỂM THỬ API VỚI POSTMAN.....	32
Các thao tác thực hành chính với Postman:	32
CÂU HỎI VÀ BÀI TẬP Củng Cố Bài 2	33

□ Câu hỏi lý thuyết:	33
□ Bài tập thực hành:.....	33
BÀI 3: THAO TÁC CƠ SỞ DỮ LIỆU VỚI ADO.NET VÀ DAPPER.....	34
3.1. TỔNG QUAN VỀ TRUY CẬP DỮ LIỆU TRONG .NET.....	34
3.2. LÀM VIỆC VỚI ADO.NET THUẦN	35
3.2.1. Quản lý Kết nối (SqlConnection & Connection String)	35
3.2.2. Đọc dữ liệu với SqlDataReader	35
3.3. TỐI ƯU HÓA VỚI DAPPER (MICRO-ORM)	36
3.3.1. Cài đặt Package	37
3.3.2. Thực thi Query & Command với Dapper	37
3.4. QUẢN LÝ TRANSACTION VÀ BẢO MẬT HỆ THỐNG	38
3.4.1. Quản lý SqlTransaction.....	38
3.4.2. Khai thác và Phòng chống SQL Injection.....	40
3.5. TỔ CHỨC LỚP REPOSITORY PATTERN	40
Bước 1: Định nghĩa Interface.....	40
Bước 2: Triển khai Lớp cụ thể	41
Bước 3: Đăng ký DI và Inject vào Controller	42
3.6. SO SÁNH NHANH VỚI ENTITY FRAMEWORK CORE (EF CORE)	44
CÂU HỎI VÀ BÀI TẬP Củng cố BÀI 3	44
□ Câu hỏi lý thuyết:	44
□ Bài tập thực hành:.....	45
BÀI 4: AUTHENTICATION, AUTHORIZATION & SECURITY	46
4.1. KHÁI NIỆM XÁC THỰC VÀ PHÂN QUYỀN TRONG WEB API	46
4.2. XÁC THỰC NÂNG CAO VỚI JWT (JSON WEB TOKEN).....	46

4.2.1. Cấu trúc của JWT.....	46
4.2.2. Luồng làm việc chuẩn (Access Token & Refresh Token Flow).....	47
4.2.3. Cài đặt và Triển khai JWT trong ASP.NET Core.....	47
4.3. BẢO MẬT VỚI AUTHORIZATION (POLICY & ROLE-BASED)	51
4.3.1. Role-based Authorization (Phân quyền theo Vai trò).....	51
4.3.2. Policy-based Authorization (Phân quyền theo Chính sách)	52
4.4. BẢO VỆ API KEY VÀ CẤU HÌNH CORS.....	53
4.4.1. Cấu hình CORS (Cross-Origin Resource Sharing).....	53
4.4.2. Bảo vệ Public API bằng API Key Middleware	53
4.5. LỖ HỒNG BẢO MẬT PHỔ BIẾN VÀ CÁCH PHÒNG CHỐNG	55
CÂU HỎI VÀ BÀI TẬP Củng cố Bài 4	55
□ Câu hỏi lý thuyết:	55
□ Bài tập thực hành:.....	55
BÀI 5: QUẢN LÝ LỖI, LOGGING VÀ TỐI ƯU HIỆU NĂNG	57
5.1. XỬ LÝ LỖI TOÀN CỤC (GLOBAL EXCEPTION HANDLING)	57
Triển khai Custom Exception Handler Middleware	57
5.2. GHI LOG CHUYÊN NGHIỆP VỚI SERILOG (STRUCTURED LOGGING)	60
5.2.1. Cài đặt Packages	60
5.2.2. Cấu hình Serilog trong Program.cs	61
5.2.3. Sử dụng Structured Logging trong Service / Controller	62
5.3. CHIẾN LƯỢC CACHING NÂNG CAO HIỆU NĂNG	62
5.3.1. In-Memory Cache (Bộ nhớ trong)	63
5.3.2. Distributed Cache với Redis (Bộ nhớ phân tán)	64
5.4. RESPONSE COMPRESSION & RATE LIMITING	66

5.4.1. Response Compression (Nén dữ liệu trả về).....	66
5.4.2. Rate Limiting (Giới hạn tần suất gọi API).....	67
5.5. QUẢN LÝ PHIÊN BẢN API (API VERSIONING).....	68
Cài đặt Package	68
Cấu hình URL Path Versioning	68
Triển khai Controller theo từng Version.....	69
5.6. GIÁM SÁT SỨC KHỎE ỨNG DỤNG (HEALTH CHECKS).....	69
CÂU HỎI VÀ BÀI TẬP Củng cố Bài 5	70
□ Câu hỏi lý thuyết:	70
□ Bài tập thực hành:.....	70
BÀI 6: KIẾN TRÚC ỨNG DỤNG VÀ HỆ THỐNG NÂNG CAO.....	71
6.1. TỔ CHỨC MÃ NGUỒN THEO CLEAN ARCHITECTURE	71
6.1.1. Bốn tầng cốt lõi trong Clean Architecture	71
6.1.2. Thực hành cấu hình giải pháp Clean Architecture (.NET Solution).....	72
6.2. KIẾN TRÚC MONOLITHIC VS MICROSERVICES	75
6.3. GIAO TIẾP GIỮA CÁC SERVICES: HTTP CLIENT & gRPC	76
6.3.1. Sử dụng IHttpConnectionFactory tối ưu kết nối HTTP	76
6.3.2. Giao tiếp siêu tốc giữa các Microservice bằng gRPC.....	77
6.4. XÂY DỰNG API GATEWAY BẰNG YARP (YET ANOTHER REVERSE PROXY)	79
Bước 1: Cài đặt Package trên dự án API Gateway	79
Bước 2: Cấu hình Routing trong appsettings.json	79
Bước 3: Đăng ký YARP trong Program.cs.....	80
6.5. ĐÓNG GÓI DOCKER VÀ TRIỂN KHAI ỨNG DỤNG.....	81

6.5.1. Viết Dockerfile Multi-Stage tối ưu dung lượng Image.....	81
6.5.2. Quản lý Đa Container với docker-compose.yml	82
CÂU HỎI VÀ BÀI TẬP Củng cố Bài 6	83
☐ Câu hỏi lý thuyết:	83
☐ Bài tập thực hành:.....	83
BÀI 7: TESTING VÀ QUY TRÌNH PHÁT TRIỂN CI/CD	84
7.1. UNIT TESTING TRONG BACKEND .NET	84
7.1.1. Cấu trúc một test case tiêu chuẩn: Mô hình AAA (Arrange - Act - Assert).....	84
7.1.2. Khởi tạo dự án Test với xUnit	84
7.2. GIẢ LẬP DỮ LIỆU VÀ PHỤ THUỘC (MOCKING) VỚI MOQ	84
Cài đặt Package Moq	84
Viết Unit Test cho Business Logic	84
7.3. INTEGRATION TESTING KIỂM THỬ TOÀN DIỆN API	87
7.3.1. Cài đặt Package WebApplicationFactory	87
7.3.2. Viết Integration Test cho API Endpoint	87
7.4. TỰ ĐỘNG HÓA CI/CD PIPELINE VỚI GITHUB ACTIONS.....	89
Cấu hình Workflow File trên GitHub (.github/workflows/ci-cd.yml).....	89
CÂU HỎI VÀ BÀI TẬP Củng cố Bài 7	91
☐ Câu hỏi lý thuyết:	91
☐ Bài tập thực hành:.....	92
BÀI 8: ỨNG DỤNG THỰC TẾ – HỆ THỐNG QUẢN LÝ BÁN HÀNG ONLINE (FINAL PROJECT)	93
8.1. TỔNG QUAN DỰ ÁN VÀ PHÂN TÍCH YÊU CẦU.....	93
Yêu cầu chức năng chính (Core Features)	93

□ Module 1 - Xác thực & Phân quyền (Auth & Identity):	93
□ Module 2 - Quản lý Sản phẩm & Danh mục (Catalog Management):	93
□ Module 3 - Quản lý Giỏ hàng & Đơn hàng (Cart & Order Processing):	93
8.2. KHỞI TẠO CẤU TRÚC DỰ ÁN (CLEAN ARCHITECTURE)	93
8.3. THIẾT KẾ CƠ SỞ DỮ LIỆU SQL SERVER	94
8.4. PHÁT TRIỂN BACKEND WEB API (ASP.NET CORE).....	96
8.4.1. Application Layer: Định nghĩa DTO & Interfaces	96
8.4.2. Infrastructure Layer: Triển khai Dapper với Transaction.....	97
8.4.3. Presentation Layer: OrdersController	100
8.5. INTEGRATION TEST CỦA DỰ ÁN	102
8.6. KẾT NỐI FRONTEND DỰ ÁN (REACTJS INTEGRATION)	103
8.7. TRIỂN KHAI VÀ ĐÓNG GÓI DOCKER TOÀN DIỆN.....	104
BÁO CÁO VÀ TIÊU CHÍ ĐÁNH GIÁ ĐỒ ÁN CUỐI KHÓA.....	106
PHỤ LỤC A. QUY ƯỚC THIẾT KẾ API ÁP DỤNG TRONG KHÓA HỌC	108
PHỤ LỤC B. CHECKLIST HOÀN THIỆN ĐỒ ÁN.....	108
PHỤ LỤC C. RUBRIC CHẤM ĐỒ ÁN CUỐI KHÓA	109
PHỤ LỤC D. THUẬT NGỮ CỐT LÕI	109
TÀI LIỆU THAM KHẢO VÀ HƯỚNG DẪN TỰ HỌC	110
PHẦN BỔ SUNG VÀ MỞ RỘNG CHUYÊN SÂU	112
BÀI 9: THIẾT KẾ API CHUYÊN NGHIỆP VÀ QUẢN LÝ HỢP ĐỒNG API.....	112
9.1. Mục tiêu bài học	112
9.2. Thiết kế tài nguyên và URI	112
9.3. Phân trang, lọc, sắp xếp và tìm kiếm.....	113
9.4. Chuẩn hóa lỗi bằng Problem Details	114

9.5. Versioning và tương thích ngược	115
9.6. Idempotency cho POST giao dịch.....	116
9.7. OpenAPI theo hướng contract-first	116
9.8. Bài tập.....	116
BÀI 10: BẢO MẬT API THEO HƯỚNG HIỆN ĐẠI.....	116
10.1. Mô hình đe dọa	116
10.2. Access token, ID token và OAuth/OIDC	117
10.3. Refresh token rotation	117
10.4. Authorization theo tài nguyên	118
10.5. Quản lý secrets	118
10.6. CORS, CSRF và token storage.....	119
10.7. Rate limiting	119
10.8. Checklist bảo mật	119
BÀI 11: ĐỘ TIN CẬY, HIỆU NĂNG VÀ KHẢ NĂNG QUAN SÁT	120
11.1. Logs, metrics và traces	120
11.2. Health checks.....	121
11.3. Caching đúng cách	121
11.4. Timeout, retry và circuit breaker	121
11.5. Tối ưu SQL và Dapper	122
11.6. Background jobs.....	122
11.7. Bài tập.....	122
BÀI 12: KIẾN TRÚC ỨNG DỤNG VÀ TÍCH HỢP HỆ THỐNG	122
12.1. Clean Architecture đúng bản chất	122
12.2. Result Pattern.....	123

12.3. Outbox Pattern.....	123
12.4. Modular monolith và microservices.....	124
12.5. Multi-tenancy	124
12.6. Upload file an toàn	124
12.7. Bài tập.....	124
BÀI 13: KIỂM THỬ, CONTAINER VÀ VẬN HÀNH PRODUCTION.....	125
13.1. Test pyramid.....	125
13.2. Integration test	125
13.3. Quản lý migration.....	125
13.4. Dockerfile .NET 10	126
13.5. CI cơ bản	126
13.6. Production readiness checklist	127
PHỤ LỤC E. LỘ TRÌNH THỰC HÀNH 12 BUỔI.....	127
PHỤ LỤC F. CÂU HỎI ÔN TẬP NÂNG CAO	128
PHỤ LỤC G. CẤU TRÚC REPOSITORY ĐỀ XUẤT	129
PHẦN BỔ SUNG NÂNG CAO III – CÁC CHỦ ĐỀ THEO YÊU CẦU THỰC TẾ DOANH NGHIỆP 2026.....	129
BÀI 14: ENTITY FRAMEWORK CORE CHUYÊN SÂU.....	129
14.1. Mục tiêu bài học	129
14.2. DbContext, DbSet và Fluent API mapping.....	130
14.3. Quản lý Migration	131
14.4. Truy vấn nâng cao và tối ưu hiệu năng	131
14.4.1. Vấn đề N+1 Query	132
14.5. So sánh sâu Dapper và EF Core: khi nào dùng cái nào.....	132

14.6. Bài tập.....	133
BÀI 15: GIAO TIẾP THỜI GIAN THỰC VÀ XỬ LÝ BẤT ĐỒNG BỘ.....	133
15.1. Mục tiêu bài học	133
15.2. SignalR – giao tiếp hai chiều thời gian thực	134
15.3. Server-Sent Events (SSE) cho luồng dữ liệu một chiều.....	134
15.4. Background Services chi tiết.....	135
15.5. Kiến trúc hướng sự kiện với Message Queue (RabbitMQ).....	136
15.6. Bài tập.....	136
BÀI 16: QUAN SÁT HỆ THỐNG NÂNG CAO VÀ VẬN HÀNH CLOUD-NATIVE.....	137
16.1. Mục tiêu bài học	137
16.2. OpenTelemetry: hợp nhất Traces, Metrics, Logs	137
16.3. Correlation ID và Distributed Tracing xuyên service	137
16.4. Đóng gói và triển khai Kubernetes cơ bản	138
16.5. Feature Flag và triển khai an toàn	139
16.6. Kiểm thử tải với k6.....	140
16.7. Bài tập.....	140
BÀI 17: BẢO VỆ DỮ LIỆU, TUÂN THỦ VÀ CÁC CHỦ ĐỀ VẬN HÀNH NÂNG CAO ..	141
17.1. Mục tiêu bài học	141
17.2. Bảo vệ dữ liệu cá nhân trong thiết kế API.....	141
17.3. Audit Logging và Soft Delete.....	141
17.4. Thiết kế Webhook an toàn.....	142
17.5. Tài liệu API nâng cao (Scalar/Redoc).....	142
17.6. Upload file lớn và xử lý bất đồng bộ.....	143
17.7. Bài tập.....	143

PHỤ LỤC H. BẢNG TỔNG HỢP CÔNG CỤ VÀ THƯ VIỆN MỞ RỘNG	143
PHỤ LỤC I. GỢI Ý ĐÁP ÁN NGẮN CHO CÂU HỎI ÔN TẬP NÂNG CAO (PHỤ LỤC F)	144
TÀI LIỆU THAM KHẢO BỔ SUNG – NGUỒN CHÍNH THỨC.....	145

BÀI 1: GIỚI THIỆU VỀ WEB API VÀ ASP.NET CORE

1.1. KHÁI NIỆM WEB API, RESTful API VÀ HTTP METHODS

1.1.1. Web API là gì?

Web API (Application Programming Interface) là giao diện lập trình ứng dụng qua môi trường Web. Nó cung cấp các phương thức (endpoints) cho phép các ứng dụng khác nhau (Web Frontend, Mobile App, IoT Device, hoặc các Service khác) có thể trao đổi dữ liệu với nhau thông qua giao thức **HTTP/HTTPS**.

Mô hình tương tác:

Client (React / iOS / Android) HTTP Request JSON / XML Web API Database SQL Query

Client (React / iOS / Android) HTTP Response JSON / XML Web API Database Data Row

1.1.2. Kiến trúc RESTful API

REST (Representational State Transfer) là một kiểu kiến trúc thiết kế mạng dành cho các hệ thống phân tán. API tuân thủ các nguyên tắc thiết kế của REST được gọi là **RESTful API**.

Các nguyên tắc cốt lõi của REST:

- **Client-Server Separation:** Tách biệt hoàn toàn giữa giao diện người dùng (Client) và logic xử lý dữ liệu (Server).
- **Stateless (Phi trạng thái):** Mỗi request từ Client gửi lên Server phải chứa đầy đủ thông tin để Server hiểu và xử lý. Server không lưu trữ bất kỳ thông tin ngữ cảnh nào của Client giữa các request.
- **Cacheable:** Response trả về phải định rõ dữ liệu đó có được lưu bộ nhớ đệm (cache) ở Client hay không.
- **Uniform Interface:** Giao diện nhất quán thông qua việc sử dụng đúng Chuẩn URI, HTTP Methods và Media Types (thường là JSON).

1.1.3. Các HTTP Methods phổ biến

RESTful API sử dụng các phương thức HTTP để đại diện cho thao tác trên tài nguyên (Resource):

HTTP Method	Mục đích	Tính chất Idempotent*
GET	Truy vấn / Lấy danh sách hoặc chi tiết tài nguyên	Có
POST	Tạo mới một tài nguyên	Không
PUT	Cập nhật toàn bộ tài nguyên (hoặc tạo mới nếu chưa có)	Có
PATCH	Cập nhật một phần của tài nguyên	Không
DELETE	Xóa một tài nguyên	Có

***Idempotent (Tính trùng lặp):** Một phương thức có tính chất Idempotent nghĩa là khi bạn thực thi request đó 1 lần hay N lần liên tiếp với cùng dữ liệu, trạng thái của hệ thống trên Server vẫn thu được kết quả giống hệt nhau.

1.2. BEST PRACTICES TRONG THIẾT KẾ RESTful API

- **Sử dụng Danh từ số nhiều cho Endpoint:**
 - ✓ ✗ *Không tốt:* GET /api/getProducts, POST /api/createProduct
 - ✓ ☑ *Đúng chuẩn REST:* GET /api/products, POST /api/products
- **Thể hiện quan hệ phân cấp qua URL:**
 - ✓ Lấy danh sách đơn hàng của người dùng ID = 10: GET /api/users/10/orders
- **Sử dụng đúng HTTP Status Codes:**
 - ✓ **2xx (Success):** 200 OK, 201 Created (dùng cho POST), 204 No Content (dùng cho DELETE/PUT thành công không trả về body).
 - ✓ **4xx (Client Errors):** 400 Bad Request (sai định dạng dữ liệu), 401 Unauthorized (chưa xác thực), 403 Forbidden (không có quyền), 404 Not Found (không tìm thấy tài nguyên).
 - ✓ **5xx (Server Errors):** 500 Internal Server Error (lỗi hệ thống).

- **Luôn sử dụng định dạng JSON:** Thiết lập Content-Type: application/json trong Header của Request và Response.

1.3. CÁC CHUẨN GIAO TIẾP: REST vs GraphQL vs gRPC

Tiêu chí	RESTful API	GraphQL	gRPC
Giao thức mạng	HTTP/1.1 hoặc HTTP/2	HTTP/1.1	HTTP/2
Định dạng dữ liệu	JSON, XML	JSON	Protocol Buffers (Binary)
Cơ chế lấy dữ liệu	Có thể bị Over-fetching / Under-fetching	Client truy vấn chính xác trường dữ liệu cần	Định nghĩa trước qua file .proto
Hiệu năng	Trung bình - Khá	Trung bình	Cực cao (Serialization gọn và nhanh)
Trường hợp sử dụng	API công cộng (Public API), Web/Mobile phổ thông	Hệ thống Frontend cần linh hoạt query	Giao tiếp nội bộ giữa các Microservices

1.4. CÀI ĐẶT MÔI TRƯỜNG PHÁT TRIỂN

Để lập trình Web API với ASP.NET Core, cần chuẩn bị các công cụ sau:

- **.NET SDK (Software Development Kit):** Cài đặt phiên bản LTS mới nhất từ trang chủ Microsoft.
 - ✓ Kiểm tra phiên bản bằng Terminal / Command Prompt:
dotnet --version

- **IDE / Code Editor:**
 - ✓ **Visual Studio:** Chọn workload *Web Development*.
 - ✓ **VS Code:** Cài đặt các Extension: *C# Dev Kit, REST Client*.
- **Database & Quản lý DB:** SQL Server Express / Developer Edition & SQL Server Management Studio (SSMS).
- **API Testing Tool:** Postman hoặc Bruno.

1.5. TỔNG QUAN KIẾN TRÚC ASP.NET CORE

Một dự án Web API trong ASP.NET Core có cấu trúc mã nguồn tối giản nhưng rất mạnh mẽ.

1.5.1. Cấu trúc thư mục tiêu chuẩn

MyWebApiProject/

```

|— Controllers/                # Chứa các Controller xử lý Request
|   └─ WeatherForecastController.cs
|— Properties/
|   └─ launchSettings.json      # Cấu hình môi trường chạy (Port, SSL...)
|— appsettings.json            # File cấu hình chuỗi kết nối DB, JWT Key...
|— Program.cs                  # Entry point: Khởi tạo Host, Middleware, DI
└─ MyWebApiProject.csproj      # File quản lý dependencies & NuGet packages

```

1.5.2. File Program.cs - Trái tim của ứng dụng

Từ .NET 6 trở đi, file Program.cs sử dụng cú pháp Top-level statements, gộp việc cấu hình **Services (DI)** và **HTTP Pipeline (Middleware)** vào cùng một file.

```

var builder = WebApplication.CreateBuilder(args);
// 1. DĂNG KÝ CÁC DỊCH VỤ VÀO CONTAINER (DEPENDENCY INJECTION)
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
var app = builder.Build();
// 2. CẤU HÌNH HTTP REQUEST PIPELINE (MIDDLEWARE)
if (app.Environment.IsDevelopment())

```

```

{
    app.UseSwagger();
    app.UseSwaggerUI();
}
app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers(); // Ánh xạ các controller vào routing system
app.Run(); // Khởi chạy web application

```

1.6. MIDDLEWARE VÀ PIPELINE XỬ LÝ REQUEST

1.6.1. Middleware là gì?

Middleware là các đoạn code được sắp xếp nối tiếp nhau thành một chuỗi gọi là **HTTP Request Pipeline**. Khi một Request gửi tới, nó sẽ đi qua từng Middleware để xử lý (xác thực, logging, routing...) trước khi tới Controller. Sau khi Controller xử lý xong, Response sẽ quay ngược lại qua chuỗi Middleware đó để trả về cho Client.

Request ---> [Middleware 1] ---> [Middleware 2] ---> [Controller / Action]

Response <--- [Middleware 1] <--- [Middleware 2] <--- [Return Result]

1.6.2. Hướng dẫn tự tạo Custom Middleware

Ví dụ: Tạo một Middleware để ghi lại thời gian xử lý của mỗi Request (Performance Logging).

Bước 1: Viết lớp Middleware

```

using System.Diagnostics;

public class RequestTimingMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILogger<RequestTimingMiddleware> _logger;

    public RequestTimingMiddleware(RequestDelegate next,
    ILogger<RequestTimingMiddleware> logger)
    {
        _next = next;
        _logger = logger;
    }
}

```

```

    }

    public async Task InvokeAsync(HttpContext context)
    {
        var stopwatch = Stopwatch.StartNew();
        // Chuyển Request sang Middleware tiếp theo trong pipeline
        await _next(context);
        stopwatch.Stop();
        var elapsedMilliseconds = stopwatch.ElapsedMilliseconds;
        _logger.LogInformation("HTTP {Method} {Path} xử lý trong {Elapsed} ms",
            context.Request.Method,
            context.Request.Path,
            elapsedMilliseconds);
    }
}

```

Bước 2: Đăng ký Middleware trong Program.cs

```

var app = builder.Build();
// Đăng ký Custom Middleware của chúng ta vào Pipeline
app.UseMiddleware<RequestTimingMiddleware>();
app.UseHttpsRedirection();
app.MapControllers();
app.Run();

```

1.7. DEPENDENCY INJECTION (DI) CƠ BẢN

1.7.1. Khái niệm IoC và Dependency Injection

- **Inversion of Control (IoC):** Nguyên lý đảo ngược quyền kiểm soát. Thay vì một lớp tự khởi tạo đối tượng phụ thuộc của nó bằng từ khóa new, quyền khởi tạo đó được giao cho một khung làm việc (Framework) bên ngoài.

- **Dependency Injection (DI):** Là một kỹ thuật triển khai IoC. Các phụ thuộc (Dependencies) sẽ được "bơm" (inject) vào lớp sử dụng chúng, thông qua Constructor.

1.7.2. Vòng đời của Service (Service Lifetimes) trong ASP.NET Core

Khi đăng ký một Service vào IoC Container trong Program.cs, chúng ta phải chọn 1 trong 3 vòng đời sau:

- **Transient (AddTransient<IService, Service>()):**
 - ✓ Được khởi tạo **mỗi lần được yêu cầu** (Every time requested).
 - ✓ Phù hợp cho các Service nhẹ, không lưu trạng thái (Stateless).
- **Scoped (AddScoped<IService, Service>()):**
 - ✓ Được khởi tạo **một lần cho mỗi HTTP Request**.
 - ✓ Tất cả các thao tác trong cùng 1 request sẽ dùng chung đối tượng này.
 - ✓ Đây là lựa chọn tiêu chuẩn cho các lớp xử lý nghiệp vụ (Business Service), Repository kết nối Database.
- **Singleton (AddSingleton<IService, Service>()):**
 - ✓ Được khởi tạo **duy nhất một lần** khi ứng dụng bắt đầu chạy và dùng chung cho toàn bộ ứng dụng.
 - ✓ Phù hợp cho Caching, In-memory state, Configuration settings.

1.7.3. Minh họa code thực hành Dependency Injection

1. Khai báo Interface và Class triển khai:

```
public interface IDateTimeService
{
    DateTime GetCurrentTime();
}
public class DateTimeService : IDateTimeService
{
    public DateTime GetCurrentTime()
    {
        return DateTime.UtcNow;
    }
}
```

```
}
```

2. Đăng ký vào Container trong Program.cs:

```
builder.Services.AddScoped<IDateTimeService, DateTimeService>();
```

3. Inject vào Controller qua Constructor:

```
using Microsoft.AspNetCore.Mvc;
[ApiController]
[Route("api/[controller]")]
public class SystemController : ControllerBase
{
    private readonly IDateTimeService _dateTimeService;
    // Inject IDateTimeService thông qua Constructor
    public SystemController(IDateTimeService dateTimeService)
    {
        _dateTimeService = dateTimeService;
    }
    [HttpGet("time")]
    public IActionResult GetSystemTime()
    {
        var currentTime = _dateTimeService.GetCurrentTime();
        return Ok(new { CurrentUtcTime = currentTime });
    }
}
```

CÂU HỎI VÀ BÀI TẬP CÙNG CÓ BÀI 1

- **Câu hỏi lý thuyết:**

- ✓ Phân biệt sự khác nhau về mục đích sử dụng và tính chất Idempotent giữa 2 phương thức HTTP: PUT và PATCH.

- ✓ Chuyện gì sẽ xảy ra nếu ta Inject một Service có vòng đời Scoped vào một Service có vòng đời Singleton? (Gợi ý: Tìm hiểu về *Captive Dependency*).

- **Bài tập thực hành:**

- ✓ Cài đặt môi trường .NET SDK và VS Code / Visual Studio trên máy cá nhân.
 - ✓ Sử dụng lệnh `dotnet new webapi -n Lesson1_Sample` để tạo một dự án ASP.NET Core Web API mới từ dòng lệnh.
 - ✓ Viết một Custom Middleware đơn giản thực hiện kiểm tra: Nếu Request gửi lên có chứa Header X-API-Key thì cho phép đi tiếp, nếu không có thì trả về ngay HTTP Status Code 401 Unauthorized.
-

BÀI 2: XÂY DỰNG WEB API CƠ BẢN VÀ XỬ LÝ REQUEST

2.1. CONTROLLER-BASED API VÀ MINIMAL APIs

Trong ASP.NET Core, bạn có thể triển khai Web API theo 2 phương pháp: **Controller-based** (truyền thống, cấu trúc rõ ràng) và **Minimal APIs** (tối giản, mỏng nhẹ, giới thiệu từ .NET 6).

2.1.1. Controller-based API

Mô hình này tách biệt các nhóm endpoint theo chức năng vào từng lớp Controller kế thừa từ ControllerBase.

```
using Microsoft.AspNetCore.Mvc;

[ApiController] // Tự động hóa kiểm tra Model state, Binding data, infer Parameter
Binding

[Route("api/[controller]")] // Cấu hình Pattern Route mặc định
public class ProductsController : ControllerBase
{
    [HttpGet]
    public IActionResult GetAll()
    {
        return Ok(new[] { "Laptop", "Mouse", "Keyboard" });
    }
}
```

- **[ApiController] Attributes:** Kích hoạt các tính năng nâng cao như bắt buộc kiểm tra ModelState, tự động gửi HTTP 400 Bad Request nếu dữ liệu sai định dạng.
- **ControllerBase vs Controller:** Web API chỉ cần kế thừa ControllerBase (chứa các phương thức xử lý HTTP & JSON). Lớp Controller mở rộng thêm hỗ trợ View (HTML/Razor) dùng cho MVC.

2.1.2. Minimal APIs

Minimal APIs cho phép bạn khai báo Endpoint trực tiếp tại Program.cs mà không cần tạo file Controller riêng biệt. Thích hợp cho Microservices hoặc các Service nhỏ gọn.

```

var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();
// Khai báo Endpoint trực tiếp
app.MapGet("/api/products", () => new[] { "Laptop", "Mouse", "Keyboard" });
app.MapGet("/api/products/{id:int}", (int id) =>
    id == 1 ? Results.Ok("Laptop") : Results.NotFound());
app.Run();

```

2.2. ROUTING VÀ DATA BINDING

2.2.1. Attribute Routing

Routing quyết định đường dẫn URL nào sẽ kích hoạt hàm (Action) nào trong Controller.

```

[Route("api/v1/categories")] // Route cấp Controller
public class CategoriesController : ControllerBase
{
    [HttpGet] // URL: GET /api/v1/categories
    public IActionResult GetCategories() => Ok();

    [HttpGet("{id:int}")] // URL: GET /api/v1/categories/5 (Route Constraint: id phải là số nguyên)
    public IActionResult GetById(int id) => Ok();

    [HttpGet("{categoryId}/products/{productId}")] // URL: GET /api/v1/categories/2/products/10
    public IActionResult GetProduct(int categoryId, int productId) => Ok();
}

```

2.2.2. Data Binding Attributes

ASP.NET Core tự động đọc dữ liệu từ HTTP Request và gán vào các tham số của phương thức thông qua các Attribute chỉ định nguồn dữ liệu:

- [FromRoute]: Lấy dữ liệu từ đường dẫn URL.
- [FromQuery]: Lấy dữ liệu từ Query String trên URL (?page=1&pageSize=10).

- [FromBody]: Lấy dữ liệu từ phần Body của Request (thường là định dạng JSON).
- [FromHeader]: Lấy dữ liệu từ HTTP Header.

Ví dụ minh họa:

```
public class ProductQueryFilter
{
    public string? SearchTerm { get; set; }
    public decimal? MinPrice { get; set; }
}

[HttpPost("{storeId:int}/create")]
public IActionResult CreateProduct(
    [FromRoute] int storeId,
    [FromQuery] ProductQueryFilter filter,
    [FromBody] CreateProductDto dto,
    [FromHeader(Name = "X-Device-Id")] string deviceId)
{
    return Ok(new { storeId, filter, dto, deviceId });
}
```

2.3. TRẢ VỀ JSON VÀ CẤU HÌNH SERIALIZATION

2.3.1. Định dạng kết quả trả về với IActionResult

Web API nên sử dụng IActionResult hoặc ActionResult<T> để linh hoạt trả về các HTTP Status Code kèm dữ liệu.

```
[HttpGet("{id:int}")]
public ActionResult<ProductDto> GetProduct(int id)
{
    var product = _productService.FindById(id);
    if (product == null)
    {
```

```

        return NotFound(new { Message = $"Không tìm thấy sản phẩm có ID = {id}" }); //
404 Not Found
    }
    return Ok(product); // 200 OK
}

```

2.3.2. Cấu hình JSON Serialization (System.Text.Json)

Bạn có thể thay đổi quy tắc biến đổi Object C# sang chuỗi JSON trong Program.cs:

```

builder.Services.AddControllers()
    .AddJsonOptions(options =>
    {
        // 1. Giữ nguyên tên thuộc tính C# (PascalCase) thay vì mặc định camelCase
        options.JsonSerializerOptions.PropertyNamingPolicy = null;

        // 2. Bỏ qua các trường có giá trị null khi trả về JSON
        options.JsonSerializerOptions.DefaultIgnoreCondition =
            System.Text.Json.Serialization.JsonIgnoreCondition.WhenWritingNull;

        // 3. Cho phép chuyển đổi chuỗi số thành kiểu int/decimal tự động
        options.JsonSerializerOptions.NumberHandling =
            System.Text.Json.Serialization.JsonNumberHandling.AllowReadingFromString;
    });

```

2.4. ĐẢM BẢO VỆ TOÀN DỮ LIỆU (DATA VALIDATION)

Kiểm soát dữ liệu đầu vào là bước quan trọng hàng đầu để bảo vệ hệ thống khỏi dữ liệu rác và lỗ hổng bảo mật.

2.4.1. Sử dụng Data Annotations

Khai báo các thuộc tính ràng buộc trực tiếp trên Data Transfer Object (DTO).

```

using System.ComponentModel.DataAnnotations;

public class CreateProductDto
{
    [Required(ErrorMessage = "Tên sản phẩm không được để trống")]
    [StringLength(100, MinimumLength = 3, ErrorMessage = "Tên sản phẩm phải từ 3 đến 100 ký tự")]
    public string Name { get; set; } = string.Empty;

    [Range(0.01, 100000.00, ErrorMessage = "Giá sản phẩm phải từ 0.01 đến 100,000")]
    public decimal Price { get; set; }

    [EmailAddress(ErrorMessage = "Email người tạo không hợp lệ")]
    public string ContactEmail { get; set; } = string.Empty;
}

```

2.4.2. Validation nâng cao với FluentValidation

Khi quy tắc kiểm tra dữ liệu trở nên phức tạp (phụ thuộc vào logic nghiệp vụ), thư viện **FluentValidation** là giải pháp tối ưu giúp tách biệt hoàn toàn code Validation ra khỏi DTO.

Bước 1: Cài đặt NuGet Package

```
dotnet add package FluentValidation.DependencyInjectionExtensions
```

Bước 2: Định nghĩa Validator Class

```

using FluentValidation;

public class CreateProductDtoValidator : AbstractValidator<CreateProductDto>
{
    public CreateProductDtoValidator()
    {
        RuleFor(x => x.Name)
            .NotEmpty().WithMessage("Tên sản phẩm bắt buộc phải có.")
    }
}

```

```

        .Length(3, 100).WithMessage("Độ dài tên từ 3-100 ký tự.");

RuleFor(x => x.Price)
    .GreaterThan(0).WithMessage("Giá phải lớn hơn 0.");

// Rule phức tạp có điều kiện
When(x => x.Price > 1000, () => {
    RuleFor(x => x.ContactEmail)
        .NotEmpty().WithMessage("Sản phẩm giá trị cao cần cung cấp Email liên hệ.");
});
}
}

```

Bước 3: Đăng ký FluentValidation trong Program.cs

```

using FluentValidation;

builder.Services.AddValidatorsFromAssemblyContaining<CreateProductDtoValidator>()
;

```

2.5. TÍCH HỢP SWAGGER / OPENAPI CHO TÀI LIỆU API

Swagger (OpenAPI) giúp tạo giao diện UI trực quan để tra cứu và chạy thử các API Endpoint.

2.5.1. Cấu hình Swagger nâng cao trong Program.cs

```

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(options =>
{
    options.SwaggerDoc("v1", new Microsoft.OpenApi.Models.OpenApiInfo
    {
        Title = "E-Commerce Web API",
        Version = "v1",
        Description = "Hệ thống API quản lý bán hàng online",
    });
});

```

```

        Contact = new Microsoft.OpenApi.Models.OpenApiContact
        {
            Name = "Bộ phận Kỹ thuật",
            Email = "dev@domain.com"
        }
    });
});

```

2.5.2. Thêm Mô tả chi tiết cho Endpoint (XML Documentation)

Mở file .csproj và bật tính năng tạo file XML Documentation:

```

<PropertyGroup>
    <GenerateDocumentationFile>true</GenerateDocumentationFile>
</PropertyGroup>

```

Cấu hình Swagger đọc file XML trong Program.cs:

```

var xmlFile =
    $"{System.Reflection.Assembly.GetExecutingAssembly().GetName().Name}.xml";
var xmlPath = Path.Combine(AppContext.BaseDirectory, xmlFile);
options.IncludeXmlComments(xmlPath);

```

Bổ sung XML comment trên Controller Action:

```

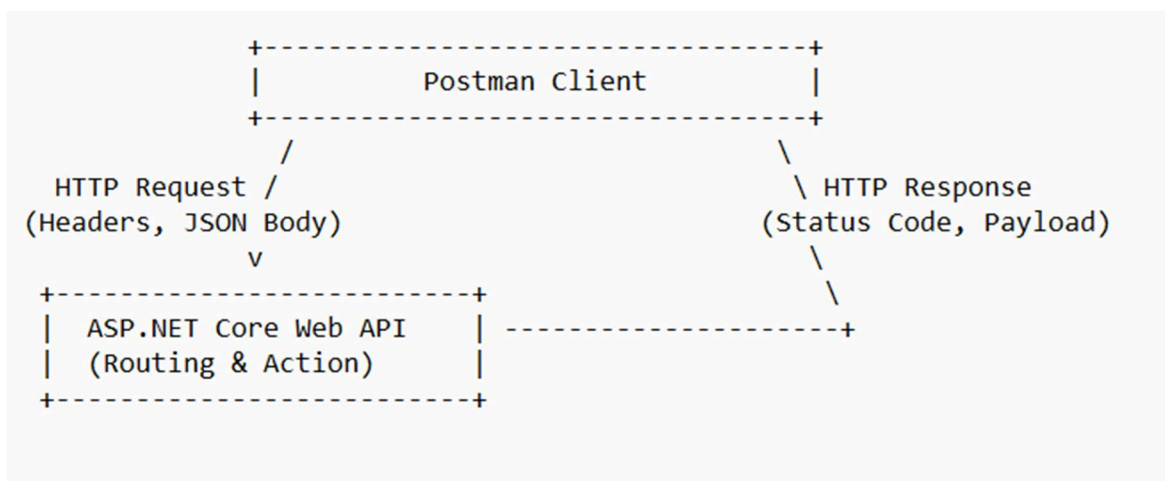
/// <summary>
/// Lấy thông tin chi tiết một sản phẩm theo ID
/// </summary>
/// <param name="id">Mã định danh duy nhất của sản phẩm</param>
/// <returns>Trả về thông tin chi tiết sản phẩm</returns>
/// <response code="200">Tìm thấy sản phẩm thành công</response>
/// <response code="404">Không tìm thấy sản phẩm trùng khớp</response>
[HttpGet("{id:int}")]

```

```
[ProducesResponseType(StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
public IActionResult GetById(int id)
{
    return Ok();
}
```

2.6. KIỂM THỬ API VỚI POSTMAN

Postman là công cụ mạnh mẽ hỗ trợ gửi HTTP Request và kiểm tra phản hồi từ API.



Các thao tác thực hành chính với Postman:

- **Tạo Collection:** Gom nhóm toàn bộ API theo tài nguyên (VD: Product Management, Order Management).
- **Quản lý Môi trường (Environments):** Thiết lập biến `{{baseUrl}}` để linh hoạt chuyển đổi giữa `https://localhost:7123` (Development) và `[https://api.mydomain.com](https://api.mydomain.com)` (Production).

Viết Test Script tự động: Chuyển sang tab **Tests** của Postman để viết script kiểm tra kết quả phản hồi tự động:

```
// Kiểm tra HTTP Status Code phải là 200 OK
pm.test("Status code is 200", function () {
    pm.response.to.have.status(200);
});
```

```
// Kiểm tra cấu trúc JSON trả về
pm.test("Response contains product ID", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData.id).to.be.a('number');
});
```

CÂU HỎI VÀ BÀI TẬP Củng cố Bài 2

- **Câu hỏi lý thuyết:**

- ✓ Khi nào nên dùng [FromQuery] và khi nào nên dùng [FromBody]?
- ✓ Ưu điểm của việc viết Validation bằng FluentValidation so với Data Annotations là gì?

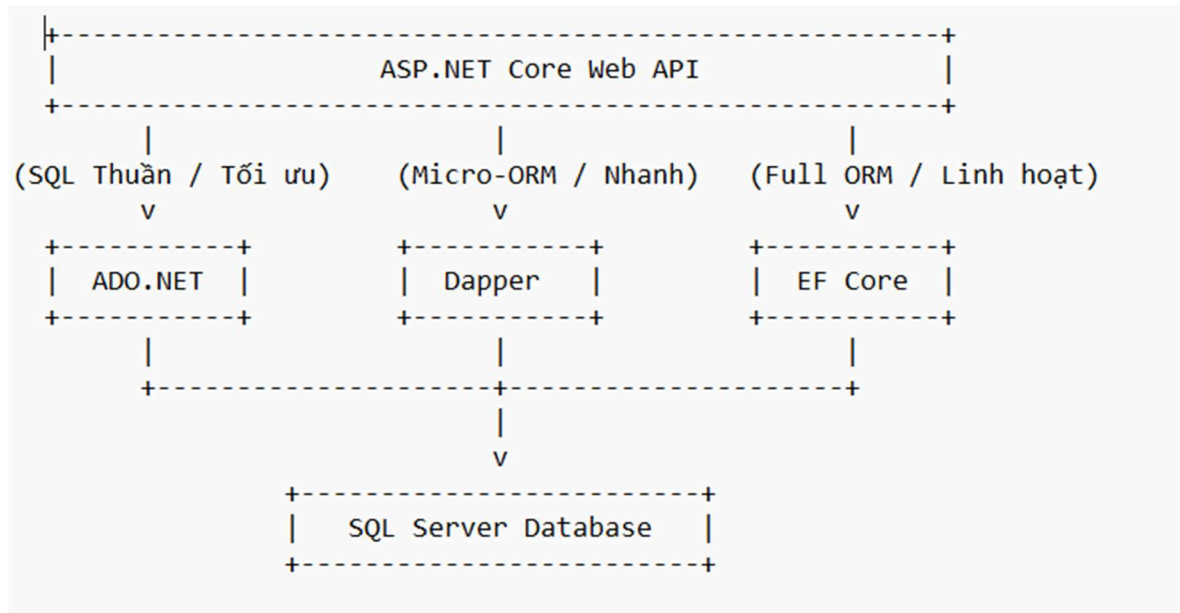
- **Bài tập thực hành:**

- ✓ Tạo một Controller tên StudentsController quản lý thông tin sinh viên bao gồm các thao tác:
 1. GET /api/students: Lấy danh sách (hỗ trợ phân trang qua Query String page và pageSize).
 2. GET /api/students/{id}: Lấy chi tiết sinh viên.
 3. POST /api/students: Tạo mới sinh viên.
- ✓ Tạo CreateStudentDto với các điều kiện: *Mã sinh viên (bắt buộc, 8 ký tự), Họ tên (bắt buộc), Tuổi (từ 18 đến 60), Email (đúng định dạng)* bằng **FluentValidation**.
- ✓ Chạy Swagger và dùng Postman thực thi lệnh POST /api/students với dữ liệu hợp lệ và không hợp lệ để kiểm tra kết quả trả về.

BÀI 3: THAO TÁC CƠ SỞ DỮ LIỆU VỚI ADO.NET VÀ DAPPER

3.1. TỔNG QUAN VỀ TRUY CẬP DỮ LIỆU TRONG .NET

Trong hệ sinh thái .NET, có 3 cách tiếp cận chính để tương tác với Cơ sở dữ liệu (CSDL) Quan hệ:



Tiêu chí	ADO.NET Thuần	Dapper (Micro-ORM)	Entity Framework Core (Full ORM)
Tốc độ thực thi	Nhanh nhất	Rất nhanh (~99% ADO.NET)	Chậm hơn (Do cần parse LINQ & change tracking)
Công sức viết code	Nhiều code boilerplate	Rất ít, đúp Mapping tự động	Ít nhất (Sinh query tự động)
Kiểm soát SQL	100% kiểm soát trực tiếp	100% kiểm soát trực tiếp	Phụ thuộc vào Generator của ORM

Tiêu chí	ADO.NET Thuần	Dapper (Micro-ORM)	Entity Framework Core (Full ORM)
Trường hợp dùng	Hệ thống cốt lõi, Query cực lớn	High-performance API, DTO Mapping	CRUD phức tạp, Nhanh chóng phát triển (Rapid Dev)

3.2. LÀM VIỆC VỚI ADO.NET THUẦN

ADO.NET cung cấp bộ các Lớp nền tảng (SqlConnection, SqlCommand, SqlDataReader) để làm việc với SQL Server.

3.2.1. Quản lý Kết nối (SqlConnection & Connection String)

Chuỗi kết nối được khai báo trong file appsettings.json:

```
{
  "ConnectionStrings": {
    "DefaultConnection":
      "Server=localhost;Database=SalesDb;Trusted_Connection=True;TrustServerCertificate=
      True;MultipleActiveResultSets=true"
  }
}
```

Lưu ý về Connection Pooling: ADO.NET tự động quản lý một tập hợp các kết nối (Pool). Lệnh connection.Open() sẽ lấy 1 kết nối có sẵn từ Pool thay vì tạo mới. Khi dùng xong, bắt buộc phải Close() hoặc Dispose() để trả kết nối lại Pool. Do đó, **luôn dùng khối using**.

3.2.2. Đọc dữ liệu với SqlDataReader

SqlDataReader đọc dữ liệu theo cơ chế **Stream một chiều (Forward-only, Read-only)** giúp tối ưu bộ nhớ RAM tối đa.

```
using Microsoft.Data.SqlClient;

public async Task<List<Product>> GetAllProductsAsync(string connectionString)
{
```

```

var products = new List<Product>();
// Sử dụng 'await using' để tự động giải phóng Connection và Command
await using var connection = new SqlConnection(connectionString);
await connection.OpenAsync();

string sql = "SELECT Id, Name, Price, CreatedAt FROM Products WHERE IsActive =
1";
await using var command = new SqlCommand(sql, connection);

await using var reader = await command.ExecuteReaderAsync();
while (await reader.ReadAsync())
{
    products.Add(new Product
    {
        Id = reader.GetInt32(reader.GetOrdinal("Id")),
        Name = reader.GetString(reader.GetOrdinal("Name")),
        Price = reader.GetDecimal(reader.GetOrdinal("Price")),
        CreatedAt = reader.GetDateTime(reader.GetOrdinal("CreatedAt"))
    });
}

return products;
}

```

3.3. TỐI ƯU HÓA VỚI DAPPER (MICRO-ORM)

Dapper là một thư viện NuGet nhẹ do Stack Overflow phát triển. Dapper giúp loại bỏ hoàn toàn đoạn code gán thủ công từng cột (`reader.GetOrdinal(...)`) bằng cách tự động Map dữ liệu trả về từ SQL vào Model C#.

3.3.1. Cài đặt Package

```
dotnet add package Dapper
dotnet add package Microsoft.Data.SqlClient
```

3.3.2. Thực thi Query & Command với Dapper

```
using Dapper;
using Microsoft.Data.SqlClient;

public class DapperProductRepository
{
    private readonly string _connectionString;
    public DapperProductRepository(IConfiguration configuration)
    {
        _connectionString = configuration.GetConnectionString("DefaultConnection");
    }

    // 1. Query danh sách
    public async Task<IEnumerable<Product>> GetAllAsync()
    {
        using var connection = new SqlConnection(_connectionString);
        string sql = "SELECT * FROM Products WHERE IsActive = 1";

        // Dapper tự động map các cột SQL trùng tên với thuộc tính C#
        return await connection.QueryAsync<Product>(sql);
    }

    // 2. Query 1 dòng với Parameterized (Chống SQL Injection)
    public async Task<Product?> GetByIdAsync(int id)
    {

```

```

        using var connection = new SqlConnection(_connectionString);
        string sql = "SELECT * FROM Products WHERE Id = @Id";

        return await connection.QueryFirstOrDefaultAsync<Product>(sql, new { Id = id });
    }

    // 3. Thực thi INSERT và trả về Identity vừa tạo
    public async Task<int> CreateAsync(Product product)
    {
        using var connection = new SqlConnection(_connectionString);
        string sql = @"
            INSERT INTO Products (Name, Price, CreatedAt, IsActive)
            VALUES (@Name, @Price, @CreatedAt, @IsActive);
            SELECT CAST(SCOPE_IDENTITY() as int);";

        return await connection.ExecuteScalarAsync<int>(sql, product);
    }
}

```

3.4. QUẢN LÝ TRANSACTION VÀ BẢO MẬT HỆ THỐNG

3.4.1. Quản lý SqlTransaction

Đảm bảo thuộc tính **ACID** (Tính nguyên tử): Tất cả thao tác thành công HOẶC không có thao tác nào được lưu.

```

public async Task<bool> CreateOrderTransactionAsync(Order order, List<OrderDetail>
details)
{
    using var connection = new SqlConnection(_connectionString);
    await connection.OpenAsync();

```

```

// Bắt đầu Transaction
using var transaction = connection.BeginTransaction();
try
{
    // 1. Insert đơn hàng
    string orderSql = @"INSERT INTO Orders (CustomerId, TotalAmount, OrderDate)
                        VALUES (@CustomerId, @TotalAmount, GETUTCDATE());
                        SELECT CAST(SCOPE_IDENTITY() as int);";

    int orderId = await connection.ExecuteScalarAsync<int>(orderSql, order,
transaction);

    // 2. Insert các mặt hàng trong đơn
    string detailSql = @"INSERT INTO OrderDetails (OrderId, ProductId, Quantity,
UnitPrice)
                        VALUES (@OrderId, @ProductId, @Quantity, @UnitPrice);";

    foreach (var detail in details)
    {
        detail.OrderId = orderId;
        await connection.ExecuteAsync(detailSql, detail, transaction);
    }

    // Commit nếu không có lỗi
    await transaction.CommitAsync();
    return true;
}
catch (Exception)
{

```

```

        // Rollback lại toàn bộ dữ liệu nếu xảy ra ngoại lệ
        await transaction.RollbackAsync();
        throw;
    }
}

```

3.4.2. Khai thác và Phòng chống SQL Injection

✗ Tuyệt đối KHÔNG cộng chuỗi SQL:

// CỰC KỲ NGUY HIỂM: Dễ bị tấn công SQL Injection

```
string sql = "SELECT * FROM Users WHERE Username = '" + userInput + "'";
```

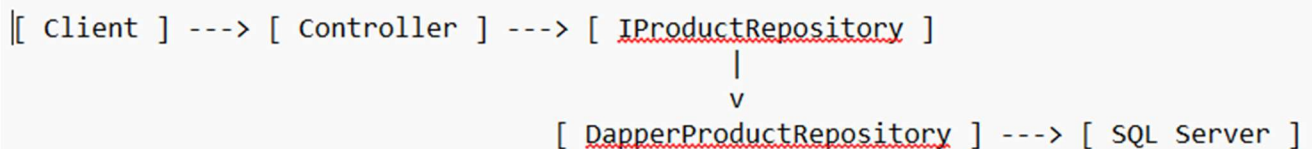
☑ Luôn luôn sử dụng Parameterized Queries:

// AN TOÀN: Driver SQL xử lý biến đầu vào dưới dạng giá trị thuần

```
string sql = "SELECT * FROM Users WHERE Username = @Username";
await connection.QueryAsync<User>(sql, new { Username = userInput });
```

3.5. TỔ CHỨC LỚP REPOSITORY PATTERN

Repository Pattern giúp trừu tượng hóa tầng Data Access, giúp Controller không cần quan tâm dữ liệu lấy từ SQL, Dapper hay Cache.



Bước 1: Định nghĩa Interface

```

public interface IProductRepository
{
    Task<IEnumerable<Product>> GetAllAsync();
    Task<Product?> GetByIdAsync(int id);
}

```

```
Task<int> CreateAsync(Product product);  
Task<bool> UpdateAsync(Product product);  
Task<bool> DeleteAsync(int id);  
}
```

Bước 2: Triển khai Lớp cụ thể

```
public class ProductRepository : IProductRepository  
{  
    private readonly string _connectionString;  
    public ProductRepository(IConfiguration configuration)  
    {  
        _connectionString = configuration.GetConnectionString("DefaultConnection");  
    }  
  
    public async Task<IEnumerable<Product>> GetAllAsync()  
    {  
        using var connection = new SqlConnection(_connectionString);  
        return await connection.QueryAsync<Product>("SELECT * FROM Products");  
    }  
  
    public async Task<Product?> GetByIdAsync(int id)  
    {  
        using var connection = new SqlConnection(_connectionString);  
        return await connection.QueryFirstOrDefaultAsync<Product>("SELECT * FROM  
Products WHERE Id = @Id", new { Id = id });  
    }  
  
    public async Task<int> CreateAsync(Product product)
```

```

    {
        using var connection = new SqlConnection(_connectionString);
        string sql = "INSERT INTO Products (Name, Price) VALUES (@Name, @Price);
SELECT CAST(SCOPE_IDENTITY() as int);";
        return await connection.ExecuteScalarAsync<int>(sql, product);
    }

    public async Task<bool> UpdateAsync(Product product)
    {
        using var connection = new SqlConnection(_connectionString);
        string sql = "UPDATE Products SET Name = @Name, Price = @Price WHERE Id =
@Id";
        int rows = await connection.ExecuteAsync(sql, product);
        return rows > 0;
    }

    public async Task<bool> DeleteAsync(int id)
    {
        using var connection = new SqlConnection(_connectionString);
        int rows = await connection.ExecuteAsync("DELETE FROM Products WHERE Id =
@Id", new { Id = id });
        return rows > 0;
    }
}

```

Bước 3: Đăng ký DI và Inject vào Controller

Trong Program.cs:

```

builder.Services.AddScoped<IProductRepository, ProductRepository>();

```

Trong ProductsController.cs:

```
[ApiController]
[Route("api/[controller]")]
public class ProductsController : ControllerBase
{
    private readonly IProductRepository _repository;
    public ProductsController(IProductRepository repository)
    {
        _repository = repository;
    }

    [HttpGet]
    public async Task<IActionResult> GetAll()
    {
        var products = await _repository.GetAllAsync();
        return Ok(products);
    }

    [HttpPost]
    public async Task<IActionResult> Create([FromBody] Product product)
    {
        int newId = await _repository.CreateAsync(product);
        return CreatedAtAction(nameof(GetById), new { id = newId }, product);
    }

    [HttpGet("{id:int}")]
    public async Task<IActionResult> GetById(int id)
    {
        var product = await _repository.GetByIdAsync(id);
```

```

        if (product == null) return NotFound();
        return Ok(product);
    }
}

```

3.6. SO SÁNH NHANH VỚI ENTITY FRAMEWORK CORE (EF CORE)

Để hình dung sự khác biệt thực tế giữa Dapper/ADO.NET và EF Core:

- **Ví dụ với Dapper:** Viết lệnh SQL trực tiếp → Kiểm soát query 100%, tốc độ đọc dữ liệu cực đỉnh.

```

var products = await connection.QueryAsync<Product>(
    "SELECT p.*, c.Name FROM Products p INNER JOIN Categories c ON p.CategoryId
    = c.Id WHERE p.Price > @Price",
    new { Price = 100 });

```

- **Ví dụ với EF Core:** Viết bằng LINQ C# → Tự động sinh SQL, linh hoạt nhưng có thêm chút chi phí CPU/Memory để dịch lệnh.

```

var products = await _context.Products
    .Include(p => p.Category)
    .Where(p => p.Price > 100)
    .ToListAsync();

```

CÂU HỎI VÀ BÀI TẬP Củng cố Bài 3

- **Câu hỏi lý thuyết:**
 - ✓ Tại sao SqlDataReader lại có hiệu năng bộ nhớ tốt hơn DataSet khi truy vấn lượng dữ liệu khổng lồ?
 - ✓ Khi nào nên dùng ExecuteScalarAsync, ExecuteAsync, và QueryAsync trong Dapper?

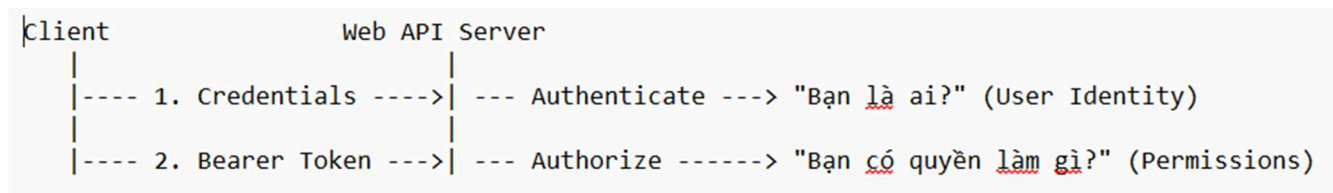
- **Bài tập thực hành:**

- ✓ Viết Stored Procedure trong SQL Server tên `sp_GetOrdersByCustomer` nhận vào `CustomerId` và trả về danh sách đơn hàng.
 - ✓ Sử dụng **Dapper** trong ASP.NET Core để gọi Stored Procedure trên thông qua `CommandType.StoredProcedure`.
 - ✓ Viết API Endpoint POST `/api/orders/transfer` thực hiện chuyển tiền/đơn hàng giữa 2 tài khoản bằng `SqlTransaction` để đảm bảo an toàn giao dịch.
-

BÀI 4: AUTHENTICATION, AUTHORIZATION & SECURITY

4.1. KHÁI NIỆM XÁC THỰC VÀ PHÂN QUYỀN TRONG WEB API

Trong phát triển Web API, hai khái niệm bảo mật cốt lõi luôn đi kèm với nhau nhưng xử lý hai nhiệm vụ hoàn toàn riêng biệt:



- **Authentication (Xác thực):** Kiểm tra danh tính của người dùng hoặc ứng dụng gửi request ("*Bạn là ai?*"). Ví dụ: Đăng nhập bằng Username/Password, OTP, hoặc Google OAuth.
- **Authorization (Phân quyền):** Xác định xem danh tính đã được kiểm tra có quyền truy cập vào tài nguyên hoặc thực thi hành động cụ thể hay không ("*Bạn có quyền làm gì?*"). Ví dụ: Chỉ tài khoản có Role Admin mới được xóa sản phẩm.

4.2. XÁC THỰC NÂNG CAO VỚI JWT (JSON WEB TOKEN)

Khác với ứng dụng Web MVC truyền thống sử dụng Session/Cookie lưu trên Server, Web API hoạt động theo nguyên tắc **Stateless**. Do đó, **JWT (JSON Web Token)** là chuẩn phổ biến nhất để xác thực.

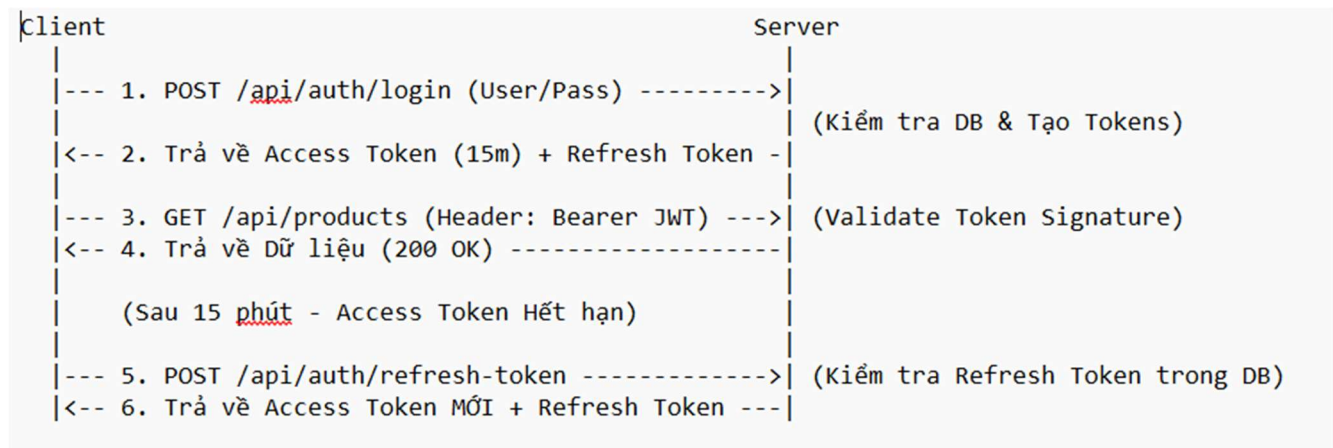
4.2.1. Cấu trúc của JWT

Một JWT là một chuỗi mã hóa gồm 3 phần phân tách nhau bởi dấu chấm (.):

Header.Payload.Signature

- **Header:** Chứa loại token (JWT) và thuật toán ký mã hóa (ví dụ: HS256).
- **Payload:** Chứa các thông tin khẳng định (**Claims**) về đối tượng (User ID, Email, Roles, Expiration Time).
- **Signature:** Chữ ký số được tạo ra bằng cách kết hợp Header + Payload với một **Secret Key** bí mật ở phía Server. Giúp Server phát hiện ngay nếu Token bị can thiệp/chỉnh sửa ở phía Client.

4.2.2. Luồng làm việc chuẩn (Access Token & Refresh Token Flow)



4.2.3. Cài đặt và Triển khai JWT trong ASP.NET Core

Bước 1: Cài đặt NuGet Packages

```
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
dotnet add package System.IdentityModel.Tokens.Jwt
```

Bước 2: Khai báo cấu hình JWT trong appsettings.json

```
{
  "JwtSettings": {
    "SecretKey": "C1S3cr3tK3yM3g4L0ngAndS3cur3N3v3rSh4r3Th1s1nPr0duct10n!",
    "Issuer": "MySalesWebApi",
    "Audience": "MySalesApiClient",
    "AccessTokenExpirationMinutes": 15
  }
}
```

Bước 3: Đăng ký JwtBearer Authentication trong Program.cs

```
using System.Text;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;

var builder = WebApplication.CreateBuilder(args);
```

```

var jwtSettings = builder.Configuration.GetSection("JwtSettings");
var secretKey = jwtSettings["SecretKey"]!;

// 1. Đăng ký Authentication Scheme
builder.Services.AddAuthentication(options =>
{
    options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
}))
.AddJwtBearer(options =>
{
    options.TokenValidationParameters = new TokenValidationParameters
    {
        ValidateIssuer = true,
        ValidateAudience = true,
        ValidateLifetime = true, // Kiểm tra thời hạn hết hạn token
        ValidateIssuerSigningKey = true, // Validation chữ ký bằng SecretKey

        ValidIssuer = jwtSettings["Issuer"],
        ValidAudience = jwtSettings["Audience"],
        IssuerSigningKey = new
SymmetricSecurityKey(Encoding.UTF8.GetBytes(secretKey)),
        ClockSkew = TimeSpan.Zero // Bỏ thời gian chênh lệch mặc định (5 phút)
    };
});

builder.Services.AddAuthorization();
var app = builder.Build();

```

```
// 2. Kích hoạt Authentication & Authorization Middleware trong Pipeline
app.UseAuthentication(); // Bắt buộc phải đặt TRƯỚC UseAuthorization
app.UseAuthorization();

app.MapControllers();
app.Run();
```

Bước 4: Viết Token Generator Service

```
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Security.Cryptography;
using System.Text;
using Microsoft.IdentityModel.Tokens;

public interface ITokenService
{
    string GenerateAccessToken(string userId, string email, IEnumerable<string> roles);
    string GenerateRefreshToken();
}

public class TokenService : ITokenService
{
    private readonly IConfiguration _config;

    public TokenService(IConfiguration config)
    {
        _config = config;
    }
}
```

```

public string GenerateAccessToken(string userId, string email, IEnumerable<string>
roles)
{
    var jwtSettings = _config.GetSection("JwtSettings");
    var secretKey = new
SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtSettings["SecretKey"]!));

    // Tạo danh sách Claims thông tin User
    var claims = new List<Claim>
    {
        new(ClaimTypes.NameIdentifier, userId),
        new(ClaimTypes.Email, email),
        new(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString())
    };

    // Bổ sung các Roles vào Claim
    claims.AddRange(roles.Select(role => new Claim(ClaimTypes.Role, role)));

    var creds = new SigningCredentials(secretKey, SecurityAlgorithms.HmacSha256);
    var expires =
DateTime.UtcNow.AddMinutes(double.Parse(jwtSettings["AccessTokenExpirationMinut
es"]!));

    var token = new JwtSecurityToken(
        issuer: jwtSettings["Issuer"],
        audience: jwtSettings["Audience"],
        claims: claims,
        expires: expires,

```

```

        signingCredentials: creds
    );

    return new JwtSecurityTokenHandler().WriteToken(token);
}

public string GenerateRefreshToken()
{
    var randomNumber = new byte[64];
    using var rng = RandomNumberGenerator.Create();
    rng.GetBytes(randomNumber);
    return Convert.ToBase64String(randomNumber);
}
}

```

4.3. BẢO MẬT VỚI AUTHORIZATION (POLICY & ROLE-BASED)

4.3.1. Role-based Authorization (Phân quyền theo Vai trò)

Sử dụng thuộc tính [Authorize] trực tiếp trên Controller hoặc Action Method.

```

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
[ApiController]
[Route("api/[controller]")]
[Authorize] // Bắt buộc phải Đăng nhập mới được gọi API này
public class OrdersController : ControllerBase
{
    [HttpGet] // Ai đăng nhập cũng xem được
    public IActionResult GetMyOrders() => Ok();

    [HttpDelete("{id}")]

```

```
[Authorize(Roles = "Admin,Manager")] // Chỉ tài khoản có Role Admin hoặc Manager
mới được xóa

public IActionResult DeleteOrder(int id) => Ok();
}
```

4.3.2. Policy-based Authorization (Phân quyền theo Chính sách)

Thích hợp cho nghiệp vụ phức tạp không thể gói gọn trong một tên Role đơn thuần.

Ví dụ: Yêu cầu người dùng phải đủ trên 18 tuổi và có thẻ VIP.

Đăng ký Policy trong Program.cs:

```
builder.Services.AddAuthorization(options =>
{
    // 1. Policy kiểm tra Claim
    options.AddPolicy("MustBeEmployee", policy =>
        policy.RequireClaim("EmployeeNumber"));

    // 2. Policy kết hợp nhiều điều kiện
    options.AddPolicy("VIPCustomerOnly", policy =>
        policy.RequireRole("Customer")
            .RequireClaim("MembershipLevel", "Gold", "Platinum"));
});
```

Sử dụng Policy trong Controller:

```
[HttpGet("vip-deals")]
[Authorize(Policy = "VIPCustomerOnly")]
public IActionResult GetVipDeals()
{
    return Ok(new[] { "Ưu đãi 50%", "Miễn phí vận chuyển" });
}
```

4.4. BẢO VỆ API KEY VÀ CẤU HÌNH CORS

4.4.1. Cấu hình CORS (Cross-Origin Resource Sharing)

Mặc định trình duyệt sẽ chặn các request JavaScript (React, Angular, Vue...) gọi tới Web API nếu khác Domain/Port. Cần mở CORS an toàn:

```
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowSpecificOrigins", policy =>
    {
        policy.WithOrigins("https://mydomain.com", "http://localhost:3000") // Chỉ cho phép domain này
        .AllowAnyHeader()
        .AllowAnyMethod()
        .AllowCredentials(); // Nếu cần dùng Cookie
    });
});

var app = builder.Build();

// Sử dụng Middleware CORS (ĐẶT TRƯỚC UseAuthorization)
app.UseCors("AllowSpecificOrigins");
```

4.4.2. Bảo vệ Public API bằng API Key Middleware

Dành cho API kết nối Service-to-Service hoặc cấp cho đối tác thứ 3.

```
public class ApiKeyMiddleware
{
    private readonly RequestDelegate _next;
    private const string APIKEYNAME = "X-API-KEY";

    public ApiKeyMiddleware(RequestDelegate next)
```

```

{
    _next = next;
}

public async Task InvokeAsync(HttpContext context, IConfiguration configuration)
{
    if (!context.Request.Headers.TryGetValue(APIKEYNAME, out var
extractedApiKey))
    {
        context.Response.StatusCode = 401;
        await context.Response.WriteAsync("Yeu cau thieu API Key.");
        return;
    }

    var apiKey = configuration.GetValue<string>("ApiKey");
    if (!apiKey!.Equals(extractedApiKey))
    {
        context.Response.StatusCode = 403;
        await context.Response.WriteAsync("API Key khong hop le.");
        return;
    }

    await _next(context);
}
}

```

4.5. LỖ HỒNG BẢO MẬT PHỔ BIẾN VÀ CÁCH PHÒNG CHỐNG

Lỗi hồng	Bản chất tấn công	Cách phòng chống triệt để
SQL Injection	Chèn các đoạn mã SQL độc hại qua tham số đầu vào để truy vấn/xóa CSDL.	Sử dụng Parameterized Query với ADO.NET/Dapper (như Bài 3) hoặc dùng ORM. Tuyệt đối không cộng chuỗi SQL.
XSS (Cross-Site Scripting)	Chèn thẻ <script> độc hại vào dữ liệu lưu trữ để chạy JS trái phép trên trình duyệt nạn nhân.	Validate/Sanitize dữ liệu đầu vào. Trả về dữ liệu chuẩn JSON (Client-side tự encode HTML khi render).
CSRF (Cross-Site Request Forgery)	Lừa trình duyệt người dùng tự gửi Request độc hại kèm Cookie xác thực.	Sử dụng JWT trong Header Authorization: Bearer <Token> thay vì lưu Authentication Token trong Cookie thuần.
Sensitive Data Exposure	Rò rỉ thông tin nhạy cảm (Mật khẩu, Key) trên đường truyền hoặc trong CSDL.	Bắt buộc HTTPS. Mã hóa Password bằng Hash 1 chiều có Salt (sử dụng BCrypt.Net hoặc PasswordHasher<T>).

CÂU HỎI VÀ BÀI TẬP Củng cố Bài 4

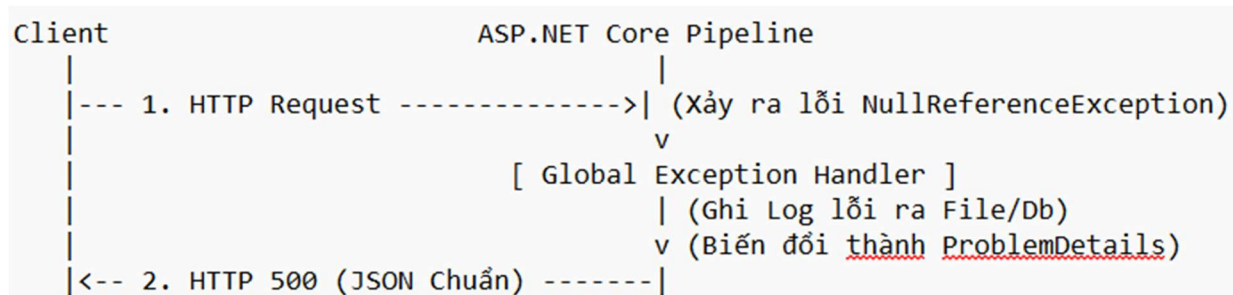
- **Câu hỏi lý thuyết:**
 - ✓ Tại sao không nên lưu các thông tin nhạy cảm (như Mật khẩu, Số thẻ tín dụng) vào phần Payload của JWT?
 - ✓ Vai trò của **Refresh Token** là gì? Tại sao không kéo dài thời gian sống của **Access Token** lên 30 ngày thay vì dùng Refresh Token?
- **Bài tập thực hành:**
 - ✓ Tạo AuthController có các API:

1. POST /api/auth/register: Đăng ký tài khoản (mã hóa mật khẩu bằng BCrypt).
 2. POST /api/auth/login: Đăng nhập, trả về Access Token (15 phút) và Refresh Token.
 3. POST /api/auth/refresh: Nhận Refresh Token để cấp lại Access Token mới.
- ✓ Tích hợp cấu hình Swagger trong Program.cs để hỗ trợ truyền JWT (Bearer Token) trực tiếp trên giao diện Swagger UI khi testing.
-

BÀI 5: QUẢN LÝ LỖI, LOGGING VÀ TỐI ƯU HIỆU NĂNG

5.1. XỬ LÝ LỖI TOÀN CỤC (GLOBAL EXCEPTION HANDLING)

Trong môi trường Production, việc để lộ nguyên văn danh sách lỗi hệ thống (*StackTrace/Exception*) ra Client vừa mất chuyên nghiệp, vừa tiềm ẩn rủi ro bảo mật lớn. Hệ thống cần có cơ chế bắt lỗi tập trung (Global Exception Handler) và trả về phản hồi theo chuẩn quốc tế **ProblemDetails** (RFC 7807).



Triển khai Custom Exception Handler Middleware

Bước 1: Tạo đối tượng phản hồi chuẩn

```
public class ErrorResponse
{
    public int StatusCode { get; set; }
    public string Title { get; set; } = string.Empty;
    public string Detail { get; set; } = string.Empty;
    public string? Instance { get; set; }
    public DateTime Timestamp { get; set; } = DateTime.UtcNow;
}
```

Bước 2: Viết Global Exception Middleware

```
using System.Net;
using System.Text.Json;

public class GlobalExceptionHandlerMiddleware
{
    private readonly RequestDelegate _next;
```

```

private readonly ILogger<GlobalExceptionMiddleware> _logger;
private readonly IHostEnvironment _env;

public GlobalExceptionMiddleware(RequestDelegate next,
ILogger<GlobalExceptionMiddleware> logger, IHostEnvironment env)
{
    _next = next;
    _logger = logger;
    _env = env;
}

public async Task InvokeAsync(HttpContext context)
{
    try
    {
        await _next(context);
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Đã xảy ra lỗi hệ thống chưa được xử lý: {Message}",
ex.Message);
        await HandleExceptionAsync(context, ex);
    }
}

private Task HandleExceptionAsync(HttpContext context, Exception exception)
{
    context.Response.ContentType = "application/json";
    var response = new ErrorResponse

```

```
{
    Instance = context.Request.Path
};

switch (exception)
{
    case KeyNotFoundException:
        context.Response.StatusCode = (int)HttpStatusCode.NotFound;
        response.StatusCode = (int)HttpStatusCode.NotFound;
        response.Title = "Resource Not Found";
        response.Detail = exception.Message;
        break;

    case UnauthorizedAccessException:
        context.Response.StatusCode = (int)HttpStatusCode.Unauthorized;
        response.StatusCode = (int)HttpStatusCode.Unauthorized;
        response.Title = "Unauthorized";
        response.Detail = exception.Message;
        break;

    case ArgumentException:
        context.Response.StatusCode = (int)HttpStatusCode.BadRequest;
        response.StatusCode = (int)HttpStatusCode.BadRequest;
        response.Title = "Bad Request";
        response.Detail = exception.Message;
        break;

    default:
        context.Response.StatusCode = (int)HttpStatusCode.InternalServerError;
```

```

        response.StatusCode = (int)HttpStatusCode.InternalServerError;
        response.Title = "Internal Server Error";
        // Trên Dev thì hiện đủ StackTrace, Production ẩn đi
        response.Detail = _env.IsDevelopment() ? exception.ToString() : "Đã có lỗi xảy
ra phía Server. Vui lòng thử lại sau.";
        break;
    }

    var jsonOptions = new JsonSerializerOptions { PropertyNamingPolicy =
JsonNamingPolicy.CamelCase };
    return context.Response.WriteAsync(JsonSerializer.Serialize(response,
jsonOptions));
}
}

```

Bước 3: Đăng ký Middleware trong Program.cs

```

var app = builder.Build();

// Đặt ở vị trí đầu tiên của Pipeline để bắt lỗi từ tất cả các Middleware phía sau
app.UseMiddleware<GlobalExceptionMiddleware>();

```

5.2. GHI LOG CHUYÊN NGHIỆP VỚI SERILOG (STRUCTURED LOGGING)

Ghi log chuỗi thuần (Console.WriteLine) rất khó tìm kiếm khi hệ thống sinh ra hàng triệu dòng log. **Serilog** giải quyết vấn đề này bằng **Structured Logging (Ghi log có cấu trúc JSON)**, cho phép truy vấn chính xác theo tham số.

5.2.1. Cài đặt Packages

```

dotnet add package Serilog.AspNetCore
dotnet add package Serilog.Sinks.File

```

```
dotnet add package Serilog.Sinks.Console
```

5.2.2. Cấu hình Serilog trong Program.cs

```
using Serilog;

// 1. Khởi tạo Serilog
Log.Logger = new LoggerConfiguration()
    .MinimumLevel.Information()
    .WriteTo.Console()
    .WriteTo.File("Logs/app_log_.txt", rollingInterval: RollingInterval.Day) // Tự động
xoay vòng file log mỗi ngày
    .CreateLogger();

try
{
    Log.Information("Đang khởi động Web API Server...");
    var builder = WebApplication.CreateBuilder(args);

    // 2. Chuyển mặc định Logger sang Serilog
    builder.Host.UseSerilog();

    builder.Services.AddControllers();
    var app = builder.Build();

    // 3. Tự động ghi Log các HTTP Request đi qua
    app.UseSerilogRequestLogging();

    app.MapControllers();
    app.Run();
}
```

```

}
catch (Exception ex)
{
    Log.Fatal(ex, "Ứng dụng bị dừng bất ngờ do lỗi!");
}
finally
{
    Log.CloseAndFlush();
}

```

5.2.3. Sử dụng Structured Logging trong Service / Controller

```

[HttpPost("process-order")]
public IActionResult ProcessOrder([FromBody] OrderRequest request)
{
    // Structured Log: Lưu dưới dạng Key-Value trong file JSON
    _logger.LogInformation("Đang xử lý đơn hàng {OrderId} cho khách hàng {CustomerId} với tổng tiền {TotalAmount}",
        request.OrderId, request.CustomerId, request.TotalAmount);

    return Ok();
}

```

5.3. CHIẾN LƯỢC CACHING NÂNG CAO HIỆU NĂNG

Caching là việc lưu tạm dữ liệu thường xuyên truy vấn vào bộ nhớ RAM tốc độ cao để giảm tải cho Database.

```

Client ----> Web API ----> [ Cache (RAM / Redis) ] --(Hit)----> Trả kết quả ngay (<5ms)
                        |
                        (Miss)
                        v
                        [ SQL Database ] -----> Đọc CSDL & Lưu lại Cache

```

5.3.1. In-Memory Cache (Bộ nhớ trong)

Thích hợp cho các ứng dụng đơn bản thể (Single Instance), dữ liệu dung lượng nhỏ (danh mục, tỉnh thành...).

Đăng ký & Sử dụng:

```
// Program.cs
builder.Services.AddMemoryCache();
using Microsoft.Extensions.Caching.Memory;

[ApiController]
[Route("api/[controller]")]
public class CategoriesController : ControllerBase
{
    private readonly IMemoryCache _cache;
    private readonly ICategoryRepository _repo;

    public CategoriesController(IMemoryCache cache, ICategoryRepository repo)
    {
        _cache = cache;
        _repo = repo;
    }

    [HttpGet]
    public async Task<IActionResult> GetCategories()
    {
        string cacheKey = "all_categories";

        // Thử lấy dữ liệu từ RAM
        if (!_cache.TryGetValue(cacheKey, out IEnumerable<Category>? categories))
        {
```

```

// Nếu chưa có (Cache Miss), đọc từ SQL
categories = await _repo.GetAllAsync();

// Thiết lập thời gian sống của Cache
var cacheOptions = new MemoryCacheEntryOptions()
    .SetAbsoluteExpiration(TimeSpan.FromMinutes(30)) // Hết hạn sau 30 phút
    .SetSlidingExpiration(TimeSpan.FromMinutes(5)); // Hết hạn nếu không được
gọi trong 5 phút

// Lưu lại vào RAM
_cache.Set(cacheKey, categories, cacheOptions);
}

return Ok(categories);
}
}

```

5.3.2. Distributed Cache với Redis (Bộ nhớ phân tán)

Bắt buộc khi triển khai mô hình Scale-out (nhiều bản thể API chạy song song) để dùng chung 1 kho Cache tập trung.

Bước 1: Cài đặt Package

```
dotnet add package Microsoft.Extensions.Caching.StackExchangeRedis
```

Bước 2: Cấu hình Redis trong Program.cs

```

builder.Services.AddStackExchangeRedisCache(options =>
{
    options.Configuration =
builder.Configuration.GetConnectionString("RedisConnection") ?? "localhost:6379";
    options.InstanceName = "SalesApi_";
}
)

```

```
});
```

Bước 3: Thao tác với IDistributedCache

```
using System.Text.Json;
using Microsoft.Extensions.Caching.Distributed;

public class ProductService
{
    private readonly IDistributedCache _redisCache;

    public ProductService(IDistributedCache redisCache)
    {
        _redisCache = redisCache;
    }

    public async Task<Product?> GetProductByIdAsync(int id)
    {
        string cacheKey = $"product_{id}";
        var cachedData = await _redisCache.GetStringAsync(cacheKey);

        if (!string.IsNullOrEmpty(cachedData))
        {
            return JsonSerializer.Deserialize<Product>(cachedData);
        }

        // Đọc từ Database...
        var product = new Product { Id = id, Name = "Laptop" };

        // Lưu vào Redis (Serialize sang JSON string)
```

```

var options = new DistributedCacheEntryOptions
{
    AbsoluteExpirationRelativeToNow = TimeSpan.FromHours(1)
};
await _redisCache.SetStringAsync(cacheKey, JsonSerializer.Serialize(product),
options);

return product;
}
}

```

5.4. RESPONSE COMPRESSION & RATE LIMITING

5.4.1. Response Compression (Nén dữ liệu trả về)

Giúp giảm kích thước JSON payload truyền qua băng thông mạng từ 60-80%, tăng tốc độ phản hồi cho ứng dụng Web/Mobile.

```

// Program.cs
builder.Services.AddResponseCompression(options =>
{
    options.EnableForHttps = true;

    options.Providers.Add<Microsoft.AspNetCore.ResponseCompression.BrotliCompression
Provider>();

    options.Providers.Add<Microsoft.AspNetCore.ResponseCompression.GzipCompressionP
rovider>();
});

var app = builder.Build();

```

```
app.UseResponseCompression(); // Đặt ở đầu Pipeline
```

5.4.2. Rate Limiting (Giới hạn tần suất gọi API)

Ngăn chặn các cuộc tấn công Brute-force, Spam API hoặc DDoS bằng cơ chế tích hợp từ .NET 7+.

```
using System.Threading.RateLimiting;
using Microsoft.AspNetCore.RateLimiting;

builder.Services.AddRateLimiter(options =>
{
    // Bị chối từ request thừa với HTTP Status Code 429
    options.RejectionStatusCode = StatusCodes.Status429TooManyRequests;

    // Fixed Window Limiter: Cho phép tối đa 100 requests / 1 phút per IP
    options.AddPolicy("FixedWindowPolicy", httpContext =>
        RateLimitPartition.GetFixedWindowLimiter(
            partitionKey: httpContext.Connection.RemoteIpAddress?.ToString() ??
"anonymous",
            factory: partition => new FixedWindowRateLimiterOptions
            {
                AutoReplenishment = true,
                PermitLimit = 100,
                QueueLimit = 2,
                Window = TimeSpan.FromMinutes(1)
            }
        ));
});

var app = builder.Build();
app.UseRateLimiter();
```

```
// Sử dụng Policy trên Controller
[HttpGet]
[EnableRateLimiting("FixedWindowPolicy")]
public IActionResult GetPublicData() => Ok();
```

5.5. QUẢN LÝ PHIÊN BẢN API (API VERSIONING)

Khi Web API phát triển, việc nâng cấp logic không được làm đứt gãy các ứng dụng client cũ đang chạy.

Cài đặt Package

```
dotnet add package Asp.Versioning.Http
dotnet add package Asp.Versioning.Mvc.ApiExplorer
```

Cấu hình URL Path Versioning

```
// Program.cs
builder.Services.AddApiVersioning(options =>
{
    options.DefaultApiVersion = new Asp.Versioning.ApiVersion(1, 0);
    options.AssumeDefaultVersionWhenUnspecified = true;
    options.ReportApiVersions = true;
    options.ApiVersionReader = new Asp.Versioning.UrlSegmentApiVersionReader(); //
    Đọc qua URL /api/v1/
}).AddApiExplorer(options =>
{
    options.GroupNameFormat = "'v'VVV";
    options.SubstituteApiVersionInUrl = true;
});
```

Triển khai Controller theo từng Version

```
using Asp.Versioning;

// API Version 1
[ApiController]
[ApiVersion("1.0")]
[Route("api/v{version:apiVersion}/{controller}")]
public class ProductsV1Controller : ControllerBase
{
    [HttpGet]
    public IActionResult Get() => Ok(new[] { "Product V1" });
}

// API Version 2 (Có thay đổi cấu trúc trả về)
[ApiController]
[ApiVersion("2.0")]
[Route("api/v{version:apiVersion}/{controller}")]
public class ProductsV2Controller : ControllerBase
{
    [HttpGet]
    public IActionResult Get() => Ok(new { Version = "2.0", Data = new[] { "Product V2" } });
}
```

5.6. GIÁM SÁT SỨC KHỎE ỨNG DỤNG (HEALTH CHECKS)

Health Checks cung cấp các Endpoint công khai để các hệ thống giám sát (Kubernetes, AWS Route53, Azure App Service) kiểm tra xem API và CSDL có đang hoạt động tốt không.

```
// Program.cs
builder.Services.AddHealthChecks()
```

```
.AddSqlServer(builder.Configuration.GetConnectionString("DefaultConnection"),
name: "SQL Server")

.AddRedis(builder.Configuration.GetConnectionString("RedisConnection"), name:
"Redis Cache");

var app = builder.Build();

// Endpoint kiểm tra nhanh
app.MapHealthChecks("/healthz");
```

Khi truy cập GET /healthz, hệ thống sẽ tự kiểm tra kết nối tới SQL và Redis, trả về:

- **Healthy (HTTP 200):** Tất cả dịch vụ phụ thuộc đều hoạt động hoàn hảo.
- **Unhealthy (HTTP 530/500):** Có dịch vụ bị gián đoạn kết nối.

CÂU HỎI VÀ BÀI TẬP Củng cố Bài 5

- **Câu hỏi lý thuyết:**

- ✓ Sự khác biệt lớn nhất giữa **In-Memory Cache** và **Distributed Cache (Redis)** là gì? Khi nào bắt buộc phải dùng Redis?
- ✓ Tại sao việc ghi Log bằng JSON (Structured Logging) lại quan trọng trong việc phân tích lỗi trên các hệ thống lớn?

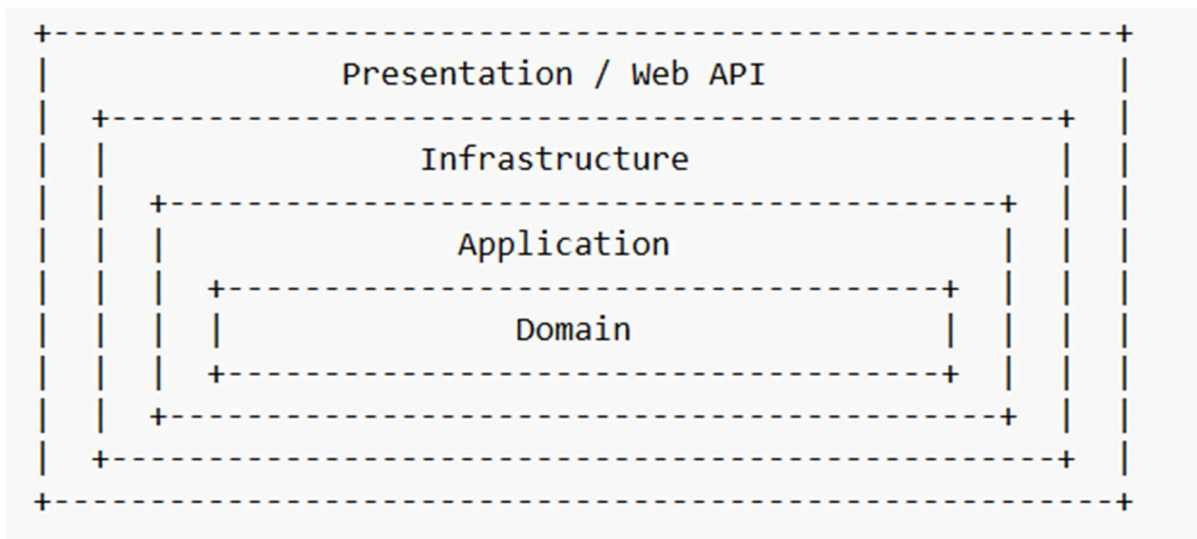
- **Bài tập thực hành:**

- ✓ Viết `GlobalExceptionHandler` hoàn chỉnh và kiểm thử bắt các lỗi: `ValidationException` (HTTP 400), `UnauthorizedAccessException` (HTTP 401), và `Exception` không xác định (HTTP 500).
- ✓ Tạo Endpoint GET `/api/v1/products/top-selling` sử dụng Redis Cache để lưu kết quả trong 15 phút. Viết thêm cơ chế **Cache Invalidation** (xóa Cache) mỗi khi có đơn hàng mới được tạo.

BÀI 6: KIẾN TRÚC ỨNG DỤNG VÀ HỆ THỐNG NÂNG CAO

6.1. TỔ CHỨC MÃ NGUỒN THEO CLEAN ARCHITECTURE

Khi ứng dụng phát triển phức tạp, việc viết toàn bộ code trong Controller hay một dự án duy nhất (Monolithic bết) sẽ khiến hệ thống trở nên cứng nhắc, khó kiểm thử và khó bảo trì. **Clean Architecture (Onion Architecture)** giải quyết vấn đề này bằng cách chia hệ thống thành các vòng tròn đồng tâm, trong đó **Domain (Nghệp vụ)** là trung tâm và độc lập hoàn toàn với công nghệ bên ngoài (Database, Framework, UI).



6.1.1. Bốn tầng cốt lõi trong Clean Architecture

- **Domain Layer (Core - Trung tâm):**
 - ✓ Chứa các Entities, Value Objects, Enums, Domain Events.
 - ✓ **Quy tắc tuyệt đối:** Không reference đến bất kỳ tầng nào khác, không phụ thuộc vào EF Core, SQL hay ASP.NET Core.
- **Application Layer (Use Cases):**
 - ✓ Chứa logic nghiệp vụ của ứng dụng (Services, DTOs, Interfaces, Validators).
 - ✓ Định nghĩa các Interfaces cho Repository, EmailService, Cache... nhưng không triển khai chi tiết.
 - ✓ Chỉ phụ thuộc vào Domain Layer.
- **Infrastructure Layer (Hạ tầng):**
 - ✓ Triển khai chi tiết các Interface từ Application (Database Access bằng Dapper/EF Core, Gửi Mail qua SMTP, Gọi Caching Redis, File Storage...).

- ✓ Phụ thuộc vào Application Layer.
- **Presentation / Web API Layer (Giao diện API):**
 - ✓ Chứa Controllers, Minimal APIs, Middlewares, Swagger Config.
 - ✓ Đóng vai trò là Entry Point, nhận HTTP Request và chuyển sang Application Layer xử lý.

6.1.2. Thực hành cấu hình giải pháp Clean Architecture (.NET Solution)

Cấu trúc Solution trên Visual Studio / VS Code:

MySalesSystem.sln

```

|— src/
|   |— Core/
|   |   |— MySalesSystem.Domain/      (Class Library)
|   |   |— MySalesSystem.Application/ (Class Library)
|   |   |— Infrastructure/
|   |       |— MySalesSystem.Infrastructure/ (Class Library)
|   |   |— Presentation/
|   |       |— MySalesSystem.Api/      (ASP.NET Core Web API)

```

1. Domain Layer (*MySalesSystem.Domain*)

```

namespace MySalesSystem.Domain.Entities;
public class Product
{
    public int Id { get; private set; }
    public string Name { get; private set; } = string.Empty;
    public decimal Price { get; private set; }
    public int StockQuantity { get; private set; }

    // Domain Logic phương thức bảo vệ trạng thái Entity
    public Product(string name, decimal price, int stockQuantity)
    {

```

```

        if (price <= 0) throw new ArgumentException("Giá sản phẩm phải lớn hơn 0");

        Name = name;
        Price = price;
        StockQuantity = stockQuantity;
    }

    public void ReduceStock(int quantity)
    {
        if (quantity > StockQuantity)
            throw new InvalidOperationException("Số lượng tồn kho không đủ.");

        StockQuantity -= quantity;
    }
}

```

2. Application Layer (*MySalesSystem.Application*)

Định nghĩa Interface trong Application Layer:

```

namespace MySalesSystem.Application.Abstracts;

public interface IProductRepository
{
    Task<Domain.Entities.Product?> GetByIdAsync(int id);
    Task UpdateAsync(Domain.Entities.Product product);
}

public interface IOrderService
{
    Task<bool> PlaceOrderAsync(int productId, int quantity);
}

```

```
}
```

Triển khai Use Case Service:

```
namespace MySalesSystem.Application.Services;

using MySalesSystem.Application.Abstracts;

public class OrderService : IOrderService
{
    private readonly IProductRepository _productRepository;

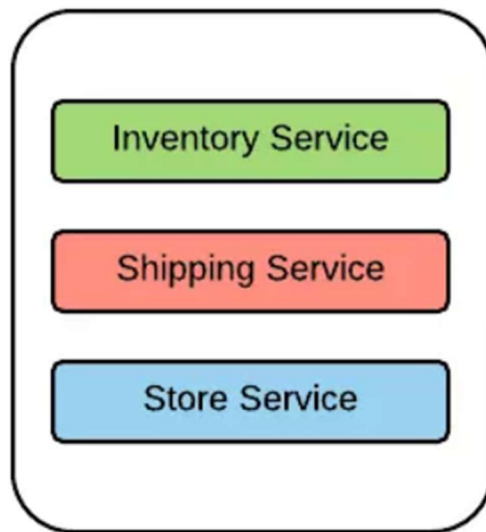
    public OrderService(IProductRepository productRepository)
    {
        _productRepository = productRepository;
    }

    public async Task<bool> PlaceOrderAsync(int productId, int quantity)
    {
        var product = await _productRepository.GetByIdAsync(productId)
            ?? throw new KeyNotFoundException("Không tìm thấy sản phẩm");

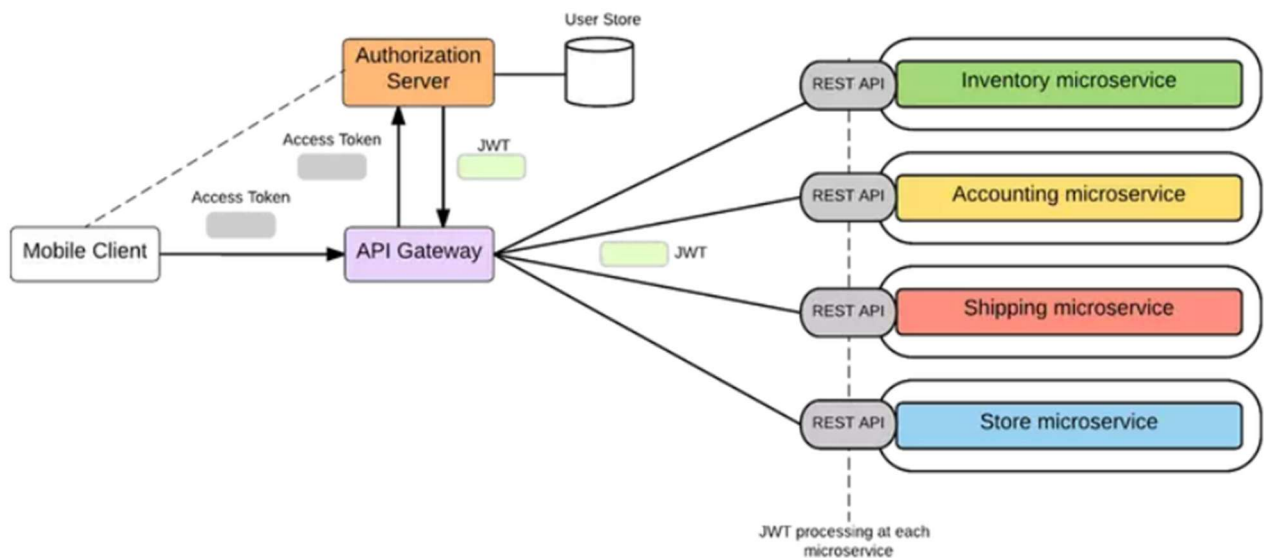
        // Gọi Domain Logic
        product.ReduceStock(quantity);

        await _productRepository.UpdateAsync(product);
        return true;
    }
}
```

6.2. KIẾN TRÚC MONOLITHIC VS MICROSERVICES



Kiến trúc: MONOLITHIC



Kiến trúc: MICROSERVICES

Tiêu chí	Monolithic (Khối đồng nhất)	Microservices (Dịch vụ nhỏ)
Độ phức tạp	Thấp, dễ phát triển, dễ test & deploy ban đầu	Rất cao, cần kiến thức DevOps, Service Discovery

Tiêu chí	Monolithic (Khối đồng nhất)	Microservices (Dịch vụ nhỏ)
Cơ sở dữ liệu	1 CSDL chung	Mỗi Service có CSDL riêng biệt (Database-per-service)
Khả năng mở rộng (Scaling)	Scale toàn bộ ứng dụng (tốn tài nguyên)	Scale riêng lẻ từng Service bị nghẽn (tối ưu chi phí)
Mức độ ảnh hưởng lỗi	1 lỗi nghiêm trọng có thể sập toàn bộ App	Lỗi ở 1 Service không làm ngắt kết nối các Service khác
Khuyến dùng khi	Dự án vừa & nhỏ, Team dưới 10-15 devs	Hệ thống lớn, traffic cực cao, Team đông phân chia độc lập

6.3. GIAO TIẾP GIỮA CÁC SERVICES: HTTP CLIENT & gRPC

Trong kiến trúc Microservices hoặc khi gọi các API bên thứ 3 (Payment Gateway, Viễn thông), việc quản lý kết nối HTTP đúng cách là rất quan trọng.

6.3.1. Sử dụng IHttpConnectionFactory tối ưu kết nối HTTP

Tránh lỗi **Socket Exhaustion** (cạn kiệt Socket do tự khởi tạo new HttpClient() thủ công).

Đăng ký Named / Typed Client trong Program.cs:

```
builder.Services.AddHttpClient("PaymentGateway", client =>
{
    client.BaseAddress = new Uri("https://api.paymentprovider.com/v1/");
    client.Timeout = TimeSpan.FromSeconds(10);
    client.DefaultRequestHeaders.Add("Accept", "application/json");
});
```

Inject và sử dụng trong Service:

```
public class PaymentService
{
```

```

private readonly IHttpClientFactory _httpClientFactory;

public PaymentService(IHttpClientFactory httpClientFactory)
{
    _httpClientFactory = httpClientFactory;
}

public async Task<bool> ProcessPaymentAsync(decimal amount)
{
    var client = _httpClientFactory.CreateClient("PaymentGateway");
    var response = await client.PostAsJsonAsync("charge", new { Amount = amount });

    return response.IsSuccessStatusCode;
}
}

```

6.3.2. Giao tiếp siêu tốc giữa các Microservice bằng gRPC

gRPC là khung làm việc RPC (Remote Procedure Call) mã nguồn mở của Google, chạy trên giao thức **HTTP/2** và định dạng dữ liệu bằng **Protocol Buffers (Binary)** giúp tốc độ nhanh gấp 7-10 lần so với REST JSON.

Bước 1: Định nghĩa Contract bằng file .proto (product.proto)

```

syntax = "proto3";

option csharp_namespace = "MySalesSystem.Grpc";

service ProductGrpc {
    rpc GetProductStock (ProductStockRequest) returns (ProductStockResponse);
}

```

```

message ProductStockRequest {
    int32 productId = 1;
}

message ProductStockResponse {
    int32 productId = 1;
    int32 stockQuantity = 2;
    bool isAvailable = 3;
}

```

Bước 2: Triển khai Service gRPC Server trong .NET

```

using Grpc.Core;
using MySalesSystem.Grpc;

public class ProductGrpcService : ProductGrpc.ProductGrpcBase
{
    public override Task<ProductStockResponse> GetProductStock(ProductStockRequest request, ServerCallContext context)
    {
        // Giả lập đọc DB
        var response = new ProductStockResponse
        {
            ProductId = request.ProductId,
            StockQuantity = 50,
            IsAvailable = true
        };

        return Task.FromResult(response);
    }
}

```

```
}
```

6.4. XÂY DỰNG API GATEWAY BẰNG YARP (YET ANOTHER REVERSE PROXY)

API Gateway đóng vai trò là cửa ngõ duy nhất tiếp nhận request từ Client, sau đó điều hướng (Routing) tới các Microservices phía sau, đồng thời xử lý Authentication, Rate Limiting tập trung.

Microsoft phát triển **YARP** là thư viện API Gateway hiệu năng cao dành riêng cho .NET.

Bước 1: Cài đặt Package trên dự án API Gateway

```
dotnet add package Yarp.ReverseProxy
```

Bước 2: Cấu hình Routing trong appsettings.json

```
{
  "ReverseProxy": {
    "Routes": {
      "product-route": {
        "ClusterId": "product-cluster",
        "Match": {
          "Path": "/api/products/{**catch-all}"
        }
      },
      "order-route": {
        "ClusterId": "order-cluster",
        "Match": {
          "Path": "/api/orders/{**catch-all}"
        }
      }
    },
    "Clusters": {
      "product-cluster": {
```

```

    "Destinations": {
      "destination1": {
        "Address": "https://localhost:7001/"
      }
    },
    "order-cluster": {
      "Destinations": {
        "destination1": {
          "Address": "https://localhost:7002/"
        }
      }
    }
  }
}

```

Bước 3: Đăng ký YARP trong Program.cs

```

var builder = WebApplication.CreateBuilder(args);

// Đăng ký Reverse Proxy từ File Cấu hình
builder.Services.AddReverseProxy()
    .LoadFromConfig(builder.Configuration.GetSection("ReverseProxy"));

var app = builder.Build();

app.MapReverseProxy(); // Kích hoạt Middleware Routing

```

```
app.Run();
```

6.5. ĐÓNG GÓI DOCKER VÀ TRIỂN KHAI ỨNG DỤNG

Docker đóng gói ứng dụng .NET cùng môi trường chạy (Runtime, OS dependencies) thành một **Container** duy nhất, đảm bảo chạy giống hệt nhau trên mọi môi trường (Dev, Staging, Production).

6.5.1. Viết Dockerfile Multi-Stage tối ưu dung lượng Image

Tạo file tên Dockerfile (không có đuôi file) nằm ở thư mục gốc dự án:

```
# STAGE 1: Build & Publish
```

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
```

```
WORKDIR /src
```

```
# Copy csproj và restore dependencies (Tối ưu Docker Cache)
```

```
COPY ["MySalesSystem.Api/MySalesSystem.Api.csproj", "MySalesSystem.Api/"]
```

```
RUN dotnet restore "MySalesSystem.Api/MySalesSystem.Api.csproj"
```

```
# Copy toàn bộ code và publish
```

```
COPY . .
```

```
WORKDIR "/src/MySalesSystem.Api"
```

```
RUN dotnet publish "MySalesSystem.Api.csproj" -c Release -o /app/publish
```

```
/p:UseAppHost=false
```

```
# STAGE 2: Runtime (Dung lượng siêu nhẹ ~200MB)
```

```
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS final
```

```
WORKDIR /app
```

```
COPY --from=build /app/publish .
```

```
EXPOSE 8080
```

```
ENV ASPNETCORE_URLS=http://+:8080
```

```
ENTRYPOINT ["dotnet", "MySalesSystem.Api.dll"]
```

6.5.2. Quản lý Đa Container với docker-compose.yml

Chạy đồng thời **Web API + SQL Server + Redis** chỉ với 1 câu lệnh.

Tạo file docker-compose.yml:

```
version: '3.8'

services:
  webapi:
    build:
      context: .
      dockerfile: Dockerfile
    ports:
      - "5000:8080"
    environment:
      - ASPNETCORE_ENVIRONMENT=Production
      - ConnectionStrings__DefaultConnection=Server=sqlserver;Database=SalesDb;User
Id=sa;Password=Your_Password123!;TrustServerCertificate=True;
      - ConnectionStrings__RedisConnection=redis:6379
    depends_on:
      - sqlserver
      - redis

  sqlserver:
    image: mcr.microsoft.com/mssql/server:2022-latest
    environment:
      - ACCEPT_EULA=Y
      - MSSQL_SA_PASSWORD=Your_Password123!
```

```
ports:
  - "1433:1433"

redis:
  image: redis:alpine
  ports:
    - "6379:6379"
```

Các lệnh Docker thao tác thực tế:

Build và khởi chạy toàn bộ hệ thống ở chế độ ngầm (Background)

`docker compose up -d --build`

Xem log chạy của API

`docker compose logs -f webapi`

Dừng và xóa toàn bộ Containers

`docker compose down`

CÂU HỎI VÀ BÀI TẬP CÙNG CỐ BÀI 6

• Câu hỏi lý thuyết:

- ✓ Tại sao tầng Domain trong Clean Architecture lại **không được chứa** các thư viện liên quan đến CSDL (như EF Core hay Dapper)?
- ✓ Khi nào nên lựa chọn gRPC thay vì RESTful JSON API cho việc truyền tải dữ liệu giữa các dịch vụ?

• Bài tập thực hành:

- ✓ Tổ chức lại cấu trúc mã nguồn một module bất kỳ theo 4 dự án Clean Architecture (.Domain, .Application, .Infrastructure, .Api).
- ✓ Viết file Dockerfile và docker-compose.yml để đóng gói dự án Web API cùng một Database SQL Server, chạy thử thành công trên Docker Desktop.

BÀI 7: TESTING VÀ QUY TRÌNH PHÁT TRIỂN CI/CD

7.1. UNIT TESTING TRONG BACKEND .NET

Unit Test (Kiểm thử đơn vị) là việc kiểm tra từng đơn vị mã nguồn nhỏ nhất (thường là một phương thức hoặc một lớp) trong môi trường cô lập, đảm bảo logic xử lý đúng như thiết kế mà không phụ thuộc vào các yếu tố bên ngoài như CSDL hay Network.

7.1.1. Cấu trúc một test case tiêu chuẩn: Mô hình AAA (Arrange - Act - Assert)

- **Arrange (Chuẩn bị):** Khởi tạo dữ liệu đầu vào, cài đặt đối tượng test và giả lập các phụ thuộc.
- **Act (Thực thi):** Gọi phương thức/chức năng cần kiểm thử.
- **Assert (Xác nhận):** So sánh kết quả trả về thực tế với kết quả kỳ vọng.

7.1.2. Khởi tạo dự án Test với xUnit

Tạo dự án xUnit Test bằng dòng lệnh và reference tới dự án Application / API:

Tạo dự án test

```
dotnet new xunit -n MySalesSystem.UnitTests
```

Thêm reference tới dự án cần kiểm thử

```
dotnet add MySalesSystem.UnitTests reference src/Core/MySalesSystem.Application
```

7.2. GIẢ LẬP DỮ LIỆU VÀ PHỤ THUỘC (MOCKING) VỚI MOQ

Khi viết Unit Test cho một Service (ví dụ OrderService), chúng ta không muốn kết nối thật tới SQL Server. Thư viện **Moq** giúp tạo ra các đối tượng giả lập (Mock Objects) thay thế cho Repository hay các External API.

Cài đặt Package Moq

```
dotnet add MySalesSystem.UnitTests package Moq
```

Viết Unit Test cho Business Logic

Ví dụ kiểm thử phương thức PlaceOrderAsync thuộc OrderService:

```
using Moq;  
using MySalesSystem.Application.Abstracts;
```

```

using MySalesSystem.Application.Services;
using MySalesSystem.Domain.Entities;
using Xunit;

namespace MySalesSystem.UnitTests.Services;

public class OrderServiceTests
{
    private readonly Mock<IProductRepository> _mockProductRepo;
    private readonly OrderService _orderService;

    public OrderServiceTests()
    {
        // 1. Tạo đối tượng Mock cho Interface IProductRepository
        _mockProductRepo = new Mock<IProductRepository>();

        // 2. Inject đối tượng giả lập vào Service cần test
        _orderService = new OrderService(_mockProductRepo.Object);
    }

    [Fact]
    public async Task
PlaceOrderAsync_WithValidStock_ShouldReturnTrueAndReduceStock()
    {
        // --- ARRANGE ---
        int productId = 10;
        int initialStock = 50;
        int orderQuantity = 5;
        var existingProduct = new Product("Laptop Dell", 1500, initialStock);
    }

```

```

// Giả lập: Khi Repo gọi GetByIdAsync(10) -> Trả về existingProduct
_mockProductRepo
    .Setup(repo => repo.GetByIdAsync(productId))
    .ReturnsAsync(existingProduct);

// --- ACT ---
bool result = await _orderService.PlaceOrderAsync(productId, orderQuantity);

// --- ASSERT ---
Assert.True(result); // Xác nhận kết quả trả về là true
Assert.Equal(45, existingProduct.StockQuantity); // Xác nhận số lượng tồn kho giảm
đúng còn 45

// Xác nhận phương thức UpdateAsync của Repo được gọi đúng 1 lần
_mockProductRepo.Verify(repo => repo.UpdateAsync(existingProduct),
Times.Once);
}

[Fact]
public async Task
PlaceOrderAsync_WhenProductNotFound_ShouldThrowKeyNotFoundException()
{
    // --- ARRANGE ---
    int nonExistentProductId = 999;

    _mockProductRepo
        .Setup(repo => repo.GetByIdAsync(nonExistentProductId))
        .ReturnsAsync((Product?)null);

```

```

// --- ACT & ASSERT ---
await Assert.ThrowsAsync<KeyNotFoundException>(() =>
    _orderService.PlaceOrderAsync(nonExistentProductId, 2));
}
}

```

7.3. INTEGRATION TESTING KIỂM THỬ TOÀN DIỆN API

Khác với Unit Test, **Integration Test (Kiểm thử tích hợp)** khởi tạo toàn bộ Pipeline xử lý HTTP Request (Routing, Middlewares, Validation, Controllers) để kiểm tra luồng hoạt động thực tế từ đầu đến cuối.

7.3.1. Cài đặt Package WebApplicationFactory

```
dotnet add package Microsoft.AspNetCore.Mvc.Testing
```

7.3.2. Viết Integration Test cho API Endpoint

Sử dụng WebApplicationFactory<Program> để khởi tạo một In-Memory Test Server chạy ngầm:

```

using System.Net;
using System.Net.Http.Json;
using Microsoft.AspNetCore.Mvc.Testing;
using MySalesSystem.Domain.Entities;
using Xunit;

namespace MySalesSystem.IntegrationTests;

public class ProductsApiTests : IClassFixture<WebApplicationFactory<Program>>
{
    private readonly HttpClient _client;
}

```

```

public ProductsApiTests(WebApplicationFactory<Program> factory)
{
    // Tạo HttpClient ảo kết nối trực tiếp vào In-Memory Test Server
    _client = factory.CreateClient();
}

[Fact]
public async Task
GetProductById_WhenProductDoesNotExist_ReturnsNotFoundStatusCode()
{
    // ACT
    var response = await _client.GetAsync("/api/products/99999");

    // ASSERT
    Assert.Equal(HttpStatusCode.NotFound, response.StatusCode);
}

[Fact]
public async Task CreateProduct_WithInvalidData_ReturnsBadRequestStatusCode()
{
    // ARRANGE: Dữ liệu vi phạm validation (tên rỗng, giá < 0)
    var invalidProductDto = new
    {
        Name = "",
        Price = -100
    };

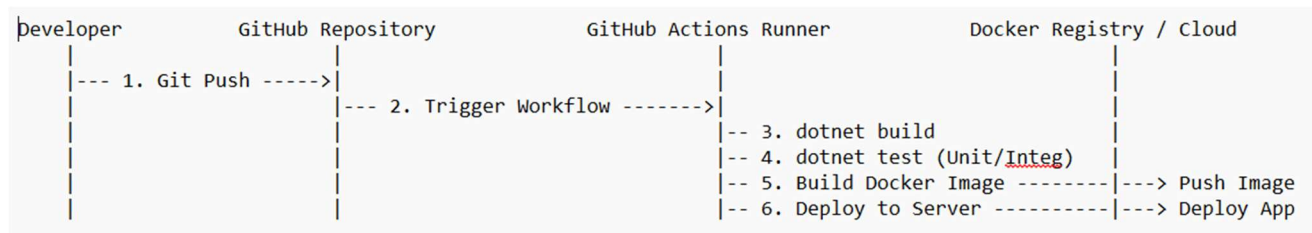
    // ACT
    var response = await _client.PostAsJsonAsync("/api/products", invalidProductDto);

```

```
// ASSERT
Assert.Equal(HttpStatusCode.BadRequest, response.StatusCode);
}
}
```

7.4. TỰ ĐỘNG HÓA CI/CD PIPELINE VỚI GITHUB ACTIONS

CI/CD (Continuous Integration / Continuous Deployment) là quy trình tự động hóa kiểm tra mã nguồn (Build & Test) mỗi khi có code mới push lên repository, đồng thời tự động đóng gói Docker Image và triển khai (Deploy) ứng dụng lên Server nếu tất cả các test case đều vượt qua.



Cấu hình Workflow File trên GitHub (.github/workflows/ci-cd.yml)

Tạo file .github/workflows/ci-cd.yml trong thư mục gốc của dự án:

```
name: Build, Test & Deploy ASP.NET Core Web API

# 1. Kích hoạt Workflow khi push hoặc tạo PR vào nhánh main
on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  build-and-test:
```

```
runs-on: ubuntu-latest

steps:
  # Bước 1: Checkout mã nguồn từ GitHub
  - name: Checkout Code
    uses: actions/checkout@v4

  # Bước 2: Thiết lập môi trường .NET SDK 8.0
  - name: Setup .NET SDK
    uses: actions/setup-dotnet@v4
    with:
      dotnet-version: '8.0.x'

  # Bước 3: Restore dependencies
  - name: Restore Dependencies
    run: dotnet restore MySalesSystem.sln

  # Bước 4: Biên dịch mã nguồn (Build)
  - name: Build Solution
    run: dotnet build MySalesSystem.sln --configuration Release --no-restore

  # Bước 5: Chạy toàn bộ Unit Test & Integration Test
  - name: Execute Tests
    run: dotnet test MySalesSystem.sln --configuration Release --no-build --verbosity
normal

deploy-docker:
  needs: build-and-test # Chỉ chạy khi job build-and-test thành công
  runs-on: ubuntu-latest
```

```

if: github.ref == 'refs/heads/main' && github.event_name == 'push'

steps:
- name: Checkout Code
  uses: actions/checkout@v4

# Bước 6: Đăng nhập vào Docker Hub
- name: Log in to Docker Hub
  uses: docker/login-action@v3
  with:
    username: ${ secrets.DOCKERHUB_USERNAME }
    password: ${ secrets.DOCKERHUB_TOKEN }

# Bước 7: Build & Push Docker Image
- name: Build & Push Docker Image
  uses: docker/build-push-action@v5
  with:
    context: .
    file: ./Dockerfile
    push: true
    tags: ${ secrets.DOCKERHUB_USERNAME }/mysalessystem-api:latest

```

CÂU HỎI VÀ BÀI TẬP Củng cố Bài 7

- **Câu hỏi lý thuyết:**

- ✓ Phân biệt sự khác nhau căn bản giữa **Unit Test** và **Integration Test** về phạm vi kiểm thử, thời gian chạy và tính độc lập với tài nguyên ngoài.
- ✓ Nguyên tắc AAA (Arrange - Act - Assert) giúp ích gì cho việc bảo trì các tập tin kiểm thử trong dự án?

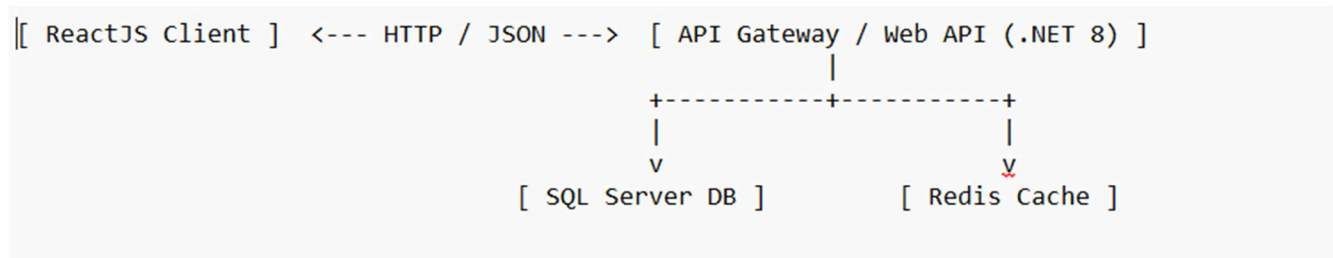
- **Bài tập thực hành:**

- ✓ Khai báo một interface `IEmailService` và viết Unit Test cho `UserService.RegisterUserAsync` bằng **Moq** để xác nhận phương thức `SendWelcomeEmailAsync` luôn được gọi khi người dùng đăng ký tài khoản thành công.
 - ✓ Tạo repository trên GitHub, đẩy mã nguồn dự án kèm file workflow `.github/workflows/ci-cd.yml` lên để quan sát quá trình chạy tự động build & test trên **GitHub Actions**.
-

BÀI 8: ỨNG DỤNG THỰC TẾ – HỆ THỐNG QUẢN LÝ BÁN HÀNG ONLINE (FINAL PROJECT)

8.1. TỔNG QUAN DỰ ÁN VÀ PHÂN TÍCH YÊU CẦU

Đề án cuối khóa là cơ hội để tổng hợp toàn bộ kiến thức từ **Bài 1 đến Bài 7**: thiết kế RESTful API theo Clean Architecture, làm việc với CSDL qua ADO.NET/Dapper, xác thực JWT & Refresh Token, tối ưu hóa với Caching/Logging, viết Unit Test, đóng gói Docker và kết nối với ứng dụng Frontend thực tế.



Yêu cầu chức năng chính (Core Features)

- **Module 1 - Xác thực & Phân quyền (Auth & Identity):**
 - ✓ Đăng ký, Đăng nhập người dùng.
 - ✓ Cấp phát JWT Access Token (15m) + Refresh Token (7 days).
 - ✓ Phân quyền theo Role: Admin, Customer.
- **Module 2 - Quản lý Sản phẩm & Danh mục (Catalog Management):**
 - ✓ API CRUD Danh mục và Sản phẩm (chỉ Admin mới có quyền CUD).
 - ✓ API xem sản phẩm công khai: Lọc theo danh mục, Phân trang (Pagination), Tìm kiếm tên sản phẩm.
 - ✓ Tích hợp Caching Redis cho danh sách sản phẩm bán chạy / danh mục.
- **Module 3 - Quản lý Giỏ hàng & Đơn hàng (Cart & Order Processing):**
 - ✓ Quản lý giỏ hàng phía Client / Redis.
 - ✓ API Đặt hàng: Xử lý Transaction (trừ tồn kho, tạo đơn hàng, tạo chi tiết đơn hàng).
 - ✓ Bảo đảm tính nguyên tử (ACID) bằng SqlTransaction hoặc Dapper Transaction.

8.2. KHỞI TẠO CẤU TRÚC DỰ ÁN (CLEAN ARCHITECTURE)

Tạo cấu trúc Solution gồm 4 dự án theo đúng chuẩn Clean Architecture đã học ở Bài 6:

ECommerceSystem.sln

```
|— src/
|   |— Core/
|   |   |— ECommerceSystem.Domain/      (Entities, Value Objects, Exceptions)
|   |   |— ECommerceSystem.Application/ (DTOs, Interfaces, Services, Validators)
|   |   |— Infrastructure/
|   |   |— ECommerceSystem.Infrastructure/ (Dapper Repositories, JWT, Redis, Serilog)
|   |   |— Presentation/
|   |   |— ECommerceSystem.Api/          (Controllers, Middlewares, Program.cs)
|   |— tests/
|       |— ECommerceSystem.UnitTests/    (xUnit & Moq Tests)
```

8.3. THIẾT KẾ CƠ SỞ DỮ LIỆU SQL SERVER

Chạy đoạn Script T-SQL sau để khởi tạo Cơ sở dữ liệu cho dự án:

```
CREATE DATABASE ECommerceDb;
GO
USE ECommerceDb;
GO

-- 1. Bảng Người dùng
CREATE TABLE Users (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    FullName NVARCHAR(100) NOT NULL,
    Email VARCHAR(100) UNIQUE NOT NULL,
    PasswordHash VARCHAR(255) NOT NULL,
    Role VARCHAR(20) NOT NULL DEFAULT 'Customer', -- Admin, Customer
    RefreshToken VARCHAR(255) NULL,
    RefreshTokenExpiryTime DATETIME2 NULL,
    CreatedAt DATETIME2 DEFAULT GETUTCDATE()
```

```
);
```

```
-- 2. Bảng Danh mục
```

```
CREATE TABLE Categories (  
    Id INT IDENTITY(1,1) PRIMARY KEY,  
    Name NVARCHAR(100) NOT NULL,  
    Description NVARCHAR(255)
```

```
);
```

```
-- 3. Bảng Sản phẩm
```

```
CREATE TABLE Products (  
    Id INT IDENTITY(1,1) PRIMARY KEY,  
    CategoryId INT NOT NULL FOREIGN KEY REFERENCES Categories(Id),  
    Name NVARCHAR(150) NOT NULL,  
    Description NVARCHAR(MAX),  
    Price DECIMAL(18,2) NOT NULL,  
    StockQuantity INT NOT NULL DEFAULT 0,  
    ImageUrl VARCHAR(500),  
    CreatedAt DATETIME2 DEFAULT GETUTCDATE()
```

```
);
```

```
-- 4. Bảng Đơn hàng
```

```
CREATE TABLE Orders (  
    Id INT IDENTITY(1,1) PRIMARY KEY,  
    UserId INT NOT NULL FOREIGN KEY REFERENCES Users(Id),  
    TotalAmount DECIMAL(18,2) NOT NULL,  
    Status NVARCHAR(50) DEFAULT N'Pending', -- Pending, Completed, Cancelled  
    ShippingAddress NVARCHAR(255) NOT NULL,  
    OrderDate DATETIME2 DEFAULT GETUTCDATE()
```

```
);

-- 5. Bảng Chi tiết đơn hàng
CREATE TABLE OrderDetails (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    OrderId INT NOT NULL FOREIGN KEY REFERENCES Orders(Id),
    ProductId INT NOT NULL FOREIGN KEY REFERENCES Products(Id),
    Quantity INT NOT NULL,
    UnitPrice DECIMAL(18,2) NOT NULL
);
```

8.4. PHÁT TRIỂN BACKEND WEB API (ASP.NET CORE)

8.4.1. Application Layer: Định nghĩa DTO & Interfaces

```
// Application/DTOs/OrderDtos.cs
namespace ECommerceSystem.Application.DTOs;

public record CreateOrderItemDto(int ProductId, int Quantity);

public record CreateOrderDto(
    string ShippingAddress,
    List<CreateOrderItemDto> Items
);

// Application/Abstracts/IOrderRepository.cs
namespace ECommerceSystem.Application.Abstracts;

public interface IOrderRepository
{
    Task<int> CreateOrderWithDetailsAsync(int userId, string shippingAddress,
```

```
List<CreateOrderItemDto> items);  
}
```

8.4.2. Infrastructure Layer: Triển khai Dapper với Transaction

Sử dụng Dapper và SqlTransaction để đảm bảo khi đặt hàng: **Nếu trừ tồn kho thất bại hoặc lỗi kết nối → Rollback toàn bộ đơn hàng.**

```
namespace ECommerceSystem.Infrastructure.Repositories;  
  
using System.Data;  
using Dapper;  
using ECommerceSystem.Application.Abstracts;  
using ECommerceSystem.Application.DTOs;  
using Microsoft.Data.SqlClient;  
using Microsoft.Extensions.Configuration;  
  
public class OrderRepository : IOrderRepository  
{  
    private readonly string _connectionString;  
  
    public OrderRepository(IConfiguration configuration)  
    {  
        _connectionString = configuration.GetConnectionString("DefaultConnection");  
    }  
  
    public async Task<int> CreateOrderWithDetailsAsync(int userId, string  
shippingAddress, List<CreateOrderItemDto> items)  
    {  
        using var connection = new SqlConnection(_connectionString);  
        await connection.OpenAsync();
```

```

// Bắt đầu Transaction
using var transaction = connection.BeginTransaction();

try
{
    decimal totalAmount = 0;

    // 1. Kiểm tra tồn kho và tính tổng tiền
    foreach (var item in items)
    {
        string productSql = "SELECT Price, StockQuantity FROM Products WHERE
Id = @Id";
        var product = await connection.QueryFirstOrDefaultAsync<(decimal Price, int
StockQuantity)>(
            productSql, new { Id = item.ProductId }, transaction);

        if (product.Equals(default) || product.StockQuantity < item.Quantity)
        {
            throw new InvalidOperationException($"Sản phẩm mã #{item.ProductId}
không đủ số lượng tồn kho.");
        }

        totalAmount += product.Price * item.Quantity;

        // 2. Trừ tồn kho
        string updateStockSql = "UPDATE Products SET StockQuantity =
StockQuantity - @Quantity WHERE Id = @Id";
        await connection.ExecuteAsync(updateStockSql, new { Quantity =

```

```

item.Quantity, Id = item.ProductId }, transaction);
    }

    // 3. Tạo Đơn hàng (Orders)
    string insertOrderSql = @"
        INSERT INTO Orders (UserId, TotalAmount, ShippingAddress, OrderDate)
        VALUES (@UserId, @TotalAmount, @ShippingAddress, GETUTCDATE());
        SELECT CAST(SCOPE_IDENTITY() as int);";

    int orderId = await connection.ExecuteScalarAsync<int>(insertOrderSql, new
    {
        UserId = userId,
        TotalAmount = totalAmount,
        ShippingAddress = shippingAddress
    }, transaction);

    // 4. Tạo Chi tiết đơn hàng (OrderDetails)
    string insertDetailSql = @"
        INSERT INTO OrderDetails (OrderId, ProductId, Quantity, UnitPrice)
        SELECT @OrderId, @ProductId, @Quantity, Price
        FROM Products WHERE Id = @ProductId;";

    foreach (var item in items)
    {
        await connection.ExecuteAsync(insertDetailSql, new
        {
            OrderId = orderId,
            ProductId = item.ProductId,
            Quantity = item.Quantity

```

```

        }, transaction);
    }

    // Commit nếu tất cả bước đều thành công
    await transaction.CommitAsync();
    return orderId;
}
catch (Exception)
{
    // Hoàn tác nếu có bất kỳ lỗi nào
    await transaction.RollbackAsync();
    throw;
}
}
}

```

8.4.3. Presentation Layer: OrdersController

```

namespace ECommerceSystem.Api.Controllers;

using System.Security.Claims;
using ECommerceSystem.Application.Abstracts;
using ECommerceSystem.Application.DTOs;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

[ApiController]
[Route("api/[controller]")]
[Authorize] // Yêu cầu đăng nhập
public class OrdersController : ControllerBase

```

```

{
    private readonly IOrderRepository _orderRepository;

    public OrdersController(IOrderRepository orderRepository)
    {
        _orderRepository = orderRepository;
    }

    /// <summary>
    /// Đặt hàng trực tuyến
    /// </summary>
    [HttpPost]
    public async Task<IActionResult> CreateOrder([FromBody] CreateOrderDto dto)
    {
        // Lấy UserId từ JWT Claim
        var userIdClaim = User.FindFirst(ClaimTypes.NameIdentifier)?.Value;
        if (string.IsNullOrEmpty(userIdClaim)) return Unauthorized();

        int userId = int.Parse(userIdClaim);

        int orderId = await _orderRepository.CreateOrderWithDetailsAsync(userId,
dto.ShippingAddress, dto.Items);

        return Ok(new
        {
            Success = true,
            Message = "Đặt hàng thành công!",
            OrderId = orderId
        });
    }
}

```

```
}  
}
```

8.5. INTEGRATION TEST CỦA DỰ ÁN

Viết Integration Test kiểm tra luồng tạo đơn hàng khi dữ liệu giỏ hàng trống hoặc thiếu Token xác thực:

```
namespace ECommerceSystem.IntegrationTests;  
  
using System.Net;  
using System.Net.Http.Json;  
using Microsoft.AspNetCore.Mvc.Testing;  
using Xunit;  
  
public class OrdersApiTests : IClassFixture<WebApplicationFactory<Program>>  
{  
    private readonly HttpClient _client;  
  
    public OrdersApiTests(WebApplicationFactory<Program> factory)  
    {  
        _client = factory.CreateClient();  
    }  
  
    [Fact]  
    public async Task CreateOrder_WithoutToken_ReturnsUnauthorized()  
    {  
        var orderDto = new { ShippingAddress = "123 Le Loi, Hanoi", Items = new  
List<object>() };  
  
        var response = await _client.PostAsJsonAsync("/api/orders", orderDto);
```

```
    Assert.Equal(HttpStatusCode.Unauthorized, response.StatusCode);  
  }  
}
```

8.6. KẾT NỐI FRONTEND DỰ ÁN (REACTJS INTEGRATION)

Dưới đây là đoạn mã ví dụ phía Frontend (ReactJS + Axios) gọi tới Backend API vừa xây dựng để xử lý đăng nhập và lưu giữ Token:

```
import axios from 'axios';  
  
const API_BASE_URL = 'http://localhost:5000/api';  
  
// Config Axios Instance có sẵn Interceptor tự gắn Token  
const apiClient = axios.create({  
  baseURL: API_BASE_URL,  
  headers: {  
    'Content-Type': 'application/json'  
  }  
});  
  
// Tự động gắn JWT Access Token vào Header mỗi Request  
apiClient.interceptors.request.use((config) => {  
  const token = localStorage.getItem('accessToken');  
  if (token) {  
    config.headers.Authorization = `Bearer ${token}`;  
  }  
  return config;  
});
```

```
// Hàm gọi API Đăng nhập
export const loginUser = async (email, password) => {
  try {
    const response = await apiClient.post('/auth/login', { email, password });
    const { accessToken, refreshToken } = response.data;

    // Lưu Token vào LocalStorage
    localStorage.setItem('accessToken', accessToken);
    localStorage.setItem('refreshToken', refreshToken);

    return response.data;
  } catch (error) {
    console.error("Đăng nhập thất bại:", error.response?.data?.message || error.message);
    throw error;
  }
};

// Hàm gọi API Đặt hàng
export const placeOrder = async (shippingAddress, items) => {
  const response = await apiClient.post('/orders', { shippingAddress, items });
  return response.data;
};
```

8.7. TRIỂN KHAI VÀ ĐÓNG GÓI DOCKER TOÀN DIỆN

Tạo file docker-compose.yml chạy hoàn chỉnh Backend API, SQL Server và Redis trên cùng một mạng ảo (Docker Network):

```
version: '3.8'

services:
```

ecommerce-api:

build:

context: .

dockerfile: Dockerfile

ports:

- "5000:8080"

environment:

- ASPNETCORE_ENVIRONMENT=Production

-

ConnectionStrings__DefaultConnection=Server=sqlserver;Database=ECommerceDb;User Id=sa;Password=SecurePassword123!;TrustServerCertificate=True;

- ConnectionStrings__RedisConnection=redis:6379

-

JwtSettings__SecretKey=C1S3cr3tK3yM3g4L0ngAndS3cur3N3v3rSh4r3Th1s1nPr0duct10n!

- JwtSettings__Issuer=ECommerceApi

- JwtSettings__Audience=ECommerceClient

depends_on:

- sqlserver

- redis

sqlserver:

image: mcr.microsoft.com/mssql/server:2022-latest

container_name: ecommerce-db

environment:

- ACCEPT_EULA=Y

- MSSQL_SA_PASSWORD=SecurePassword123!

ports:

- "1433:1433"

```

redis:
  image: redis:alpine
  container_name: ecommerce-cache
  ports:
    - "6379:6379"

networks:
  default:
    name: ecommerce-network

```

BÁO CÁO VÀ TIÊU CHÍ ĐÁNH GIÁ ĐỒ ÁN CUỐI KHÓA

Học viên sẽ trình bày đồ án trước giảng viên theo các tiêu chí sau:

Tiêu chí đánh giá	Trọng số	Yêu cầu đạt được
Kiến trúc & Code Quality	30%	Áp dụng đúng Clean Architecture, code sạch, không thừa thãi, tuân thủ nguyên lý SOLID.
Tính trọn vẹn chức năng	30%	Thực hiện đúng và đủ các luồng Auth, CRUD Sản phẩm, Caching và Đặt hàng bằng Transaction.
Bảo mật & Tối ưu	20%	Có Validation, xử lý Global Exception, JWT + Refresh Token, dùng Parameterized SQL chống SQL Injection.
Testing & Deployment	20%	Có ít nhất 3 Unit Tests/Integration Tests chạy pass và đóng gói thành công bằng

Tiêu chí đánh giá	Trọng số	Yêu cầu đạt được
		Docker Compose.

PHỤ LỤC A. QUY ƯỚC THIẾT KẾ API ÁP DỤNG TRONG KHÓA HỌC

- URI dùng danh từ số nhiều, chữ thường và dấu gạch nối khi cần; không đưa động từ CRUD vào URI.
- Mọi endpoint phải xác định rõ request DTO, response DTO, status code thành công và các trường hợp lỗi.
- Không trả trực tiếp entity CSDL ra ngoài API; sử dụng DTO để kiểm soát hợp đồng dữ liệu.
- Dữ liệu phân trang nên trả metadata gồm page, pageSize, totalItems và totalPages.
- Lỗi nghiệp vụ phải được chuẩn hóa theo ProblemDetails; không trả stack trace ra môi trường production.
- Mọi truy vấn SQL nhận dữ liệu đầu vào phải dùng parameter; không nối chuỗi SQL từ dữ liệu người dùng.
- Bí mật như JWT key, connection string và mật khẩu không commit vào source code; dùng secret/environment variable.
- Endpoint nhạy cảm phải có authentication, authorization, rate limit và audit log phù hợp.

PHỤ LỤC B. CHECKLIST HOÀN THIỆN ĐỒ ÁN

- ☐ Solution tuân thủ Dependency Rule của Clean Architecture.
- ☐ Database có script khởi tạo và dữ liệu mẫu.
- ☐ Có API đăng ký, đăng nhập, refresh token và thu hồi refresh token.
- ☐ Có phân quyền Admin/Customer và kiểm tra truy cập trái phép.
- ☐ CRUD sản phẩm/danh mục có validation, tìm kiếm và phân trang.
- ☐ Đặt hàng chạy trong transaction và kiểm tra tồn kho.
- ☐ Có global exception handler và response lỗi chuẩn hóa.
- ☐ Có structured logging; không ghi token/mật khẩu vào log.
- ☐ Có Redis cache và chiến lược invalidation rõ ràng.
- ☐ Swagger mô tả authentication và response codes.
- ☐ Có unit test cho business service và integration test cho endpoint quan trọng.

- ☐ Docker Compose chạy được API, SQL Server và Redis.
- ☐ Workflow CI thực hiện restore, build và test.
- ☐ README hướng dẫn cài đặt, cấu hình, chạy và tài khoản demo.

PHỤ LỤC C. RUBRIC CHẤM ĐỒ ÁN CUỐI KHÓA

Nhóm tiêu chí	Điểm tối đa	Đạt yêu cầu	Mức xuất sắc
Phân tích và thiết kế	15	Đúng yêu cầu, CSDL hợp lý	Mô hình rõ, quyết định thiết kế có lập luận
Chức năng nghiệp vụ	25	Các luồng chính chạy đúng	Xử lý đầy đủ biên, transaction và tính nhất quán
Kiến trúc và chất lượng mã	20	Phân tầng, DI, DTO, validation	Clean Architecture rõ, code dễ mở rộng, ít lặp
Bảo mật	15	JWT và role authorization	Refresh token an toàn, policy, secret management, audit
Hiệu năng và vận hành	10	Logging, caching, health check	Cache invalidation hợp lý, rate limit, quan sát tốt
Kiểm thử	10	Có unit/integration test	Bao phủ các luồng quan trọng và test lỗi
Triển khai và trình bày	5	Docker chạy được, demo rõ	CI/CD ổn định, README và thuyết trình chuyên nghiệp

PHỤ LỤC D. THUẬT NGỮ CỐT LÕI

Thuật ngữ	Giải thích
API	Giao diện cho phép các hệ thống phần mềm trao đổi dữ liệu hoặc gọi chức năng.
REST	Kiểu kiến trúc sử dụng tài nguyên, URI,

Thuật ngữ	Giải thích
	HTTP method và biểu diễn dữ liệu.
Idempotent	Thực hiện cùng một yêu cầu nhiều lần không làm thay đổi trạng thái cuối so với một lần.
Middleware	Thành phần nằm trong HTTP pipeline, xử lý request và response.
Dependency Injection	Cơ chế cung cấp phụ thuộc từ bên ngoài thay vì lớp tự khởi tạo.
DTO	Đối tượng dùng để trao đổi dữ liệu giữa các tầng hoặc qua API.
JWT	Định dạng token có chữ ký, thường dùng cho xác thực stateless.
Refresh Token	Thông tin có thời hạn dài hơn dùng để xin access token mới.
Dapper	Micro-ORM ánh xạ kết quả SQL sang đối tượng .NET.
Transaction	Nhóm thao tác dữ liệu được commit hoặc rollback như một đơn vị nguyên tử.
Caching	Lưu tạm dữ liệu để giảm độ trễ và tải lên nguồn dữ liệu.
CI/CD	Quy trình tự động tích hợp, kiểm thử, đóng gói và triển khai phần mềm.

TÀI LIỆU THAM KHẢO VÀ HƯỚNG DẪN TỰ HỌC

- Tài liệu chính thức của Microsoft về ASP.NET Core Web API, security, testing, performance và deployment.
- Đặc tả OpenAPI và các RFC liên quan đến HTTP semantics, status codes và Problem Details.
- Tài liệu chính thức của Dapper, Serilog, FluentValidation, Redis và xUnit.
- OWASP API Security Top 10 dùng làm checklist nhận diện rủi ro bảo mật API.

- Thực hành bổ sung: xây dựng một API khác lĩnh vực như quản lý đào tạo, kho hàng hoặc dịch vụ logistics để tái sử dụng kiến thức.

PHẦN BỔ SUNG VÀ MỞ RỘNG CHUYÊN SÂU

Phần này mở rộng giáo trình theo hướng xây dựng Web API có thể triển khai và vận hành trong môi trường thực tế. Mốc tham chiếu là ASP.NET Core trên .NET 10 LTS; người học vẫn cần kiểm tra tài liệu đúng phiên bản SDK đang sử dụng.

Lưu ý: Các đoạn tự phát hành JWT phù hợp cho học tập hoặc hệ thống khép kín. Với hệ thống công khai, nên ưu tiên OAuth 2.0/OpenID Connect và một Identity Provider chuyên dụng.

BÀI 9: THIẾT KẾ API CHUYÊN NGHIỆP VÀ QUẢN LÝ HỢP ĐỒNG API

9.1. Mục tiêu bài học

- Thiết kế tài nguyên, URI, request và response nhất quán.
- Chuẩn hóa lỗi bằng Problem Details.
- Hỗ trợ phân trang, lọc, sắp xếp và tìm kiếm an toàn.
- Hiểu versioning, backward compatibility, deprecation và idempotency.
- Xem OpenAPI như một hợp đồng có thể kiểm thử và sinh client.

9.2. Thiết kế tài nguyên và URI

Một API tốt bắt đầu từ mô hình tài nguyên thay vì bắt đầu từ Controller. Tài nguyên là đối tượng nghiệp vụ có danh tính và vòng đời rõ ràng, ví dụ products, customers, orders và payments. Endpoint mô tả tài nguyên; HTTP method mô tả thao tác.

Nhu cầu	Endpoint đề xuất	Giải thích
Lấy danh sách sản phẩm	GET /api/products	Không thay đổi trạng thái máy chủ.
Tạo sản phẩm	POST /api/products	Trả 201 Created và Location.
Cập nhật đầy đủ	PUT /api/products/{id}	Gửi biểu diễn đầy đủ.
Cập nhật một phần	PATCH /api/products/{id}	Chỉ gửi trường thay đổi.
Hủy đơn hàng	POST /api/orders/{id}/cancellations	Mô hình hóa cancellation như tài nguyên nghiệp vụ.
Đơn hàng của khách	GET /api/customers/{id}/orders	Dùng quan hệ lồng nhau vừa phải.

Không lồng URI quá sâu. Khi một tài nguyên con đã có ID duy nhất, có thể truy cập trực tiếp qua collection riêng để giảm coupling với cấu trúc phân cấp.

9.3. Phân trang, lọc, sắp xếp và tìm kiếm

Endpoint trả danh sách phải giới hạn số bản ghi ngay từ đầu. pageSize cần có giá trị mặc định và mức tối đa. Tên trường sắp xếp phải được đối chiếu với whitelist, không ghép trực tiếp vào SQL.

```
GET /api/products?page=1&pageSize=20&search=laptop
    &categoryId=5&minPrice=500&maxPrice=2000
    &sort=-price,name
```

```
public sealed record ProductQuery(
    int Page = 1,
    int PageSize = 20,
    string? Search = null,
    int? CategoryId = null,
    decimal? MinPrice = null,
    decimal? MaxPrice = null,
    string? Sort = null);
```

```
public sealed record PagedResult<T>(
    IReadOnlyList<T> Items,
    int Page,
    int PageSize,
    long TotalItems);
```

9.4. Chuẩn hóa lỗi bằng Problem Details

Mọi lỗi nên có cùng cấu trúc để frontend, mobile app và hệ thống giám sát xử lý nhất quán. Không trả stack trace, SQL, connection string hoặc thông tin nội bộ ra production. traceId dùng để đối chiếu với log server.

```
builder.Services.AddProblemDetails();
builder.Services.AddExceptionHandler<GlobalExceptionHandler>();
var app = builder.Build();
app.UseExceptionHandler();

public sealed class GlobalExceptionHandler(
    IProblemDetailsService problemDetails,
    ILogger<GlobalExceptionHandler> logger) : IExceptionHandler
{
    public async ValueTask<bool> TryHandleAsync(
        HttpContext context, Exception exception,
        CancellationToken cancellationToken)
    {
        logger.LogError(exception, "Unhandled error {TraceId}",
            context.TraceIdentifier);
        var status = exception switch
        {
            KeyNotFoundException => 404,
            ArgumentException => 400,
            UnauthorizedAccessException => 403,
            _ => 500
        };
        context.Response.StatusCode = status;
        return await problemDetails.TryWriteAsync(
            new ProblemDetailsContext
```

```

{
    HttpContext = context,
    ProblemDetails =
    {
        Status = status,
        Title = status == 500 ? "Lỗi hệ thống" : exception.Message,
        Extensions = { ["traceId"] = context.TraceIdentifier }
    }
});
}
}

```

9.5. Versioning và tương thích ngược

Versioning cần thiết khi thay đổi hợp đồng có thể làm client cũ bị lỗi. URI versioning như `/api/v1/products` để quan sát trong giảng dạy và API công khai. Cần có thời gian deprecation, tài liệu migration và ngày ngừng hỗ trợ rõ ràng.

Thay đổi	Breaking?	Hướng xử lý
Thêm trường response tùy chọn	Thường không	Client nên bỏ qua trường không biết.
Đổi tên/xóa trường	Có	Tạo version mới hoặc duy trì trường cũ tạm thời.
Đổi kiểu dữ liệu	Có	Contract mới và kế hoạch migration.
Thêm endpoint	Không	Cập nhật OpenAPI.
Thêm validation bắt buộc	Có thể	Thông báo trước và đo tác động.

9.6. Idempotency cho POST giao dịch

Nếu client gửi lại request do timeout, hệ thống có thể tạo hai đơn hàng hoặc ghi nhận thanh toán hai lần. Idempotency-Key giúp server nhận ra cùng một ý định nghiệp vụ.

1. Client tạo khóa duy nhất cho một ý định.
2. Server gắn khóa với client/user và hash request body.
3. Nếu chưa có, xử lý transaction và lưu response theo khóa.
4. Nếu đã có, trả lại kết quả trước đó.
5. Khóa có TTL và cơ chế khóa đồng thời để tránh race condition.

9.7. OpenAPI theo hướng contract-first

```
builder.Services.AddOpenApi();  
var app = builder.Build();  
if (app.Environment.IsDevelopment())  
{  
    app.MapOpenApi();  
}
```

OpenAPI cần mô tả schema, examples, security scheme, operationId và mọi response code quan trọng. Có thể dùng tài liệu này để sinh SDK, contract test và phát hiện breaking change trong CI.

9.8. Bài tập

6. Nâng cấp GET /api/products với phân trang, lọc và sort theo whitelist.
7. Chuẩn hóa lỗi bằng Problem Details có traceId.
8. Thiết kế API tạo đơn hàng có Idempotency-Key.
9. Xuất OpenAPI JSON và kiểm tra đầy đủ response schema.

BÀI 10: BẢO MẬT API THEO HƯỚNG HIỆN ĐẠI

10.1. Mô hình đe dọa

Bảo mật không chỉ là thêm [Authorize]. Cần xác định tài sản, tác nhân, bề mặt tấn công, đường dữ liệu và hậu quả. Tài sản thường gồm tài khoản, token, dữ liệu cá nhân, đơn hàng, quyền quản trị và bí mật kết nối.

Mối đe dọa	Ví dụ	Biện pháp
BOLA	User A đọc đơn hàng của User B	Kiểm tra quyền trên chính tài nguyên.
Broken authentication	Refresh token không thu hồi	Token ngắn hạn, rotation, revoke.
Excessive data exposure	Trả PasswordHash	DTO rõ ràng, allowlist trường.
Mass assignment	Client gửi IsAdmin=true	Không bind trực tiếp Entity.
Injection	SQL injection	Parameterized query và least privilege.
Resource consumption	Spam endpoint nặng	Rate limit, timeout, quota, giới hạn payload.

10.2. Access token, ID token và OAuth/OIDC

Access token dành cho API authorization; ID token dành cho client xác nhận phiên đăng nhập. API phải kiểm tra chữ ký, issuer, audience và expiration. Với production, ưu tiên OAuth 2.0/OpenID Connect thay vì tự xây token service nếu không có yêu cầu đặc biệt.

10.3. Refresh token rotation

```
public sealed class RefreshToken
{
    public Guid Id { get; init; }
    public Guid UserId { get; init; }
    public string TokenHash { get; init; } = string.Empty;
    public DateTime ExpiresAtUtc { get; init; }
    public DateTime? RevokedAtUtc { get; set; }
    public Guid? ReplacedByTokenId { get; set; }
}
```

- Không lưu refresh token dạng rõ; lưu hash.

- Khi refresh thành công, thu hồi token cũ và phát token mới.
- Nếu token đã thu hồi bị dùng lại, thu hồi cả token family.
- Logout phải thu hồi refresh token.
- Không ghi access token/refresh token đầy đủ vào log.

10.4. Authorization theo tài nguyên

Role cho biết người dùng thuộc nhóm nào, nhưng không chứng minh họ sở hữu tài nguyên.

Với đơn hàng, hồ sơ hoặc tài liệu, cần resource-based authorization.

```
public sealed class OrderOwnerRequirement : IAuthorizationRequirement;

public sealed class OrderOwnerHandler
    : AuthorizationHandler<OrderOwnerRequirement, Order>
{
    protected override Task HandleRequirementAsync(
        AuthorizationHandlerContext context,
        OrderOwnerRequirement requirement,
        Order resource)
    {
        var userId = context.User.FindFirstValue(
            ClaimTypes.NameIdentifier);
        if (userId == resource.CustomerId.ToString())
            context.Succeed(requirement);
        return Task.CompletedTask;
    }
}
```

10.5. Quản lý secrets

```
dotnet user-secrets init
dotnet user-secrets set "ConnectionStrings:DefaultConnection" "..."
dotnet user-secrets set "JwtSettings:SecretKey" "..."
```

```
builder.Services.AddOptions<JwtOptions>()
    .BindConfiguration("JwtSettings")
    .ValidateDataAnnotations()
    .ValidateOnStart();
```

Production nên dùng environment variables hoặc secret manager. Không commit secret vào Git, không copy secret vào image và cần xoay vòng bí mật định kỳ.

10.6. CORS, CSRF và token storage

CORS là chính sách của trình duyệt, không phải xác thực. Không kết hợp AllowAnyOrigin với AllowCredentials. Token trong cookie cần chống CSRF; token trong Authorization header giảm CSRF nhưng vẫn chịu rủi ro XSS nếu lưu phía client không an toàn.

10.7. Rate limiting

```
builder.Services.AddRateLimiter(options =>
{
    options.AddFixedWindowLimiter("login", limiter =>
    {
        limiter.PermitLimit = 5;
        limiter.Window = TimeSpan.FromMinutes(1);
        limiter.QueueLimit = 0;
    });
    options.RejectionStatusCode = 429;
});
app.UseRateLimiter();
app.MapPost("/api/auth/login", Login)
    .RequireRateLimiting("login");
```

10.8. Checklist bảo mật

- Bắt buộc HTTPS và HSTS phù hợp.

- Secrets ngoài source và image.
- JWT kiểm tra issuer, audience, signature, lifetime.
- Mọi endpoint theo ID kiểm tra quyền trên tài nguyên.
- Giới hạn request body, upload, pageSize và timeout.
- Không trả lỗi kỹ thuật chi tiết ra client.
- Che dữ liệu cá nhân và token trong log.
- Quét dependency và cập nhật bản vá định kỳ.

BÀI 11: ĐỘ TIN CẬY, HIỆU NĂNG VÀ KHẢ NĂNG QUAN SÁT

11.1. Logs, metrics và traces

Trụ cột	Câu hỏi	Ví dụ
Logs	Chuyện gì đã xảy ra?	Lỗi nghiệp vụ, audit.
Metrics	Hệ thống đang ở mức nào?	p95 latency, error rate, CPU.
Traces	Request đi qua đâu?	API → Redis → SQL → Payment.

Mỗi request nên có correlation/trace ID để nối log giữa middleware, controller, repository và dịch vụ ngoài.

```
app.Use(async (context, next) =>
{
    var id = context.Request.Headers["X-Correlation-Id"].FirstOrDefault()
        ?? Guid.NewGuid().ToString("N");
    context.Response.Headers["X-Correlation-Id"] = id;
    using (logger.BeginScope(new Dictionary<string, object>
        { ["CorrelationId"] = id }))
    {
        await next();
    }
});
```

11.2. Health checks

Liveness cho biết tiến trình còn sống; readiness cho biết ứng dụng đã sẵn sàng nhận traffic. Readiness có thể kiểm tra CSDL hoặc cache thiết yếu nhưng phải có timeout ngắn.

```
builder.Services.AddHealthChecks();
app.MapHealthChecks("/health/live",
    new HealthCheckOptions { Predicate = _ => false });
app.MapHealthChecks("/health/ready");
```

11.3. Caching đúng cách

Cache là bản sao, không phải nguồn dữ liệu gốc. Cache-aside: đọc cache trước; miss thì đọc CSDL và ghi cache. Sau update phải invalidation sau khi transaction commit.

Rủi ro	Biểu hiện	Xử lý
Dữ liệu cũ	Update xong vẫn thấy cũ	Xóa/cập nhật cache sau commit.
Stampede	Nhiều request cùng miss	Lock, jitter TTL, stale-while-revalidate.
Key collision	Lộ dữ liệu tenant	Key chứa tenant/user/filter/version.
Dữ liệu nhạy cảm	Rò rỉ	Phân loại, TTL ngắn, mã hóa khi cần.

11.4. Timeout, retry và circuit breaker

Chỉ retry lỗi tạm thời và thao tác an toàn hoặc idempotent. Timeout của dependency phải ngắn hơn timeout tổng. Không retry vô hạn và không retry lỗi validation.

```
builder.Services.AddHttpClient<PaymentClient>(client =>
{
    client.BaseAddress = new Uri(
        builder.Configuration["Services:Payment"]);
    client.Timeout = TimeSpan.FromSeconds(10);
});
```

```
})  
.AddStandardResilienceHandler();
```

11.5. Tối ưu SQL và Dapper

- Chỉ SELECT cột cần dùng; tránh SELECT * trong contract ổn định.
- Tạo index theo truy vấn thực tế và đọc execution plan.
- Dùng keyset pagination khi offset lớn.
- Tránh N+1 query.
- Truyền CancellationToken cho I/O.
- Không giữ transaction khi gọi dịch vụ mạng.
- Luôn dispose connection để trả về pool.

11.6. Background jobs

Không giữ HTTP request mở trong nhiều phút để tạo báo cáo lớn hoặc gửi hàng nghìn email. Endpoint nên ghi nhận job, trả 202 Accepted và cung cấp URL kiểm tra trạng thái. Job quan trọng cần persistence, retry, dead-letter và chống chạy trùng.

11.7. Bài tập

10. Thêm correlation ID vào log và response.
11. Tạo liveness/readiness endpoints.
12. Áp dụng cache-aside và invalidation.
13. Dùng typed HttpClient với resilience.
14. Chuyển tác vụ dài sang 202 Accepted + job status.

BÀI 12: KIẾN TRÚC ỨNG DỤNG VÀ TÍCH HỢP HỆ THỐNG

12.1. Clean Architecture đúng bản chất

Mục tiêu là hướng phụ thuộc đi vào trong: Domain không phụ thuộc hạ tầng; Application mô tả use case; Infrastructure triển khai kỹ thuật; API chuyển đổi HTTP. Tách nhiều project không có ý nghĩa nếu ranh giới vẫn bị xuyên thủng.

Tầng	Trách nhiệm	Không nên chứa
Domain	Entity, value object, invariant, domain event	SQL, HTTP, framework code.

Application	Use case, DTO, interface port	Controller, SqlConnection cụ thể.
Infrastructure	Dapper, SQL, Redis, email	Quyết định nghiệp vụ cốt lõi.
API	Routing, auth, serialization	SQL trực tiếp, nghiệp vụ phức tạp.

12.2. Result Pattern

Application service nên trả kết quả nghiệp vụ rõ ràng thay vì ném exception cho mọi tình huống dự kiến. Controller ánh xạ mã lỗi sang 400/404/409/422 phù hợp.

```
public sealed record Result<T>(
    bool IsSuccess, T? Value,
    string? ErrorCode, string? ErrorMessage)
{
    public static Result<T> Success(T value) =>
        new(true, value, null, null);
    public static Result<T> Failure(string code, string message) =>
        new(false, default, code, message);
}
```

12.3. Outbox Pattern

Nếu commit đơn hàng thành công nhưng publish sự kiện thất bại, các hệ thống sẽ lệch. Outbox ghi dữ liệu nghiệp vụ và bản ghi sự kiện trong cùng transaction; worker publish sau. Consumer phải idempotent.

15. INSERT Orders và INSERT OutboxMessages trong một transaction.

16. Commit transaction.

17. Worker đọc message chưa publish.

18. Publish tới broker.

19. Đánh dấu PublishedAtUtc; retry khi lỗi.

12.4. Modular monolith và microservices

Tiêu chí	Modular Monolith	Microservices
Triển khai	Một đơn vị	Nhiều đơn vị.
Transaction	ACID nội bộ dễ hơn	Thường eventual consistency.
Vận hành	Đơn giản hơn	Cần gateway, tracing, automation.
Đội nhóm	Nhỏ-vừa	Nhiều nhóm độc lập.
Rủi ro	Coupling trong code	Coupling phân tán và lỗi mạng.

Không tách microservices chỉ vì hệ thống có nhiều bảng. Chỉ tách khi ranh giới nghiệp vụ, nhu cầu mở rộng, ownership và khả năng vận hành đã rõ.

12.5. Multi-tenancy

TenantId phải đi xuyên suốt từ request context đến query và cache key. Mọi truy vấn dữ liệu tenant phải có điều kiện TenantId; integration test phải chứng minh tenant A không đọc được dữ liệu tenant B.

12.6. Upload file an toàn

- Giới hạn dung lượng, loại và số lượng file.
- Không tin Content-Type hoặc tên file từ client.
- Sinh tên lưu trữ mới và chống path traversal.
- Lưu ngoài web root hoặc object storage.
- Quét malware nếu cần.
- Streaming file lớn, không tải toàn bộ vào RAM.

12.7. Bài tập

20. Tách module Orders thành bốn tầng.
21. Dùng Result Pattern cho CancelOrder.
22. Thiết kế bảng OutboxMessages và worker giả lập.
23. Viết test tenant isolation.

BÀI 13: KIỂM THỬ, CONTAINER VÀ VẬN HÀNH PRODUCTION

13.1. Test pyramid

Loại	Mục tiêu	Phụ thuộc
Unit	Quy tắc nghiệp vụ nhỏ	Fake/mock tối thiểu.
Integration	API + DI + DB	Database test/container.
Contract	Đúng schema	OpenAPI.
End-to-end	Luồng người dùng	Toàn hệ thống.
Load	Latency, throughput	Môi trường gần production.

13.2. Integration test

Mock repository không phát hiện lỗi SQL, mapping, transaction hoặc migration. Với Dapper, integration test dùng SQL Server test database hoặc container có giá trị đặc biệt.

```
public sealed class ProductsApiTests
    : IClassFixture<WebApplicationFactory<Program>>
{
    private readonly HttpClient _client;

    public ProductsApiTests(WebApplicationFactory<Program> factory)
        => _client = factory.CreateClient();

    [Fact]
    public async Task GetById_Returns404_WhenMissing()
    {
        var response = await _client.GetAsync("/api/products/999999");
        Assert.Equal(HttpStatusCode.NotFound, response.StatusCode);
    }
}
```

13.3. Quản lý migration

- Dapper cần công cụ migration hoặc script SQL versioned.

- Không sửa migration đã chạy production; tạo migration mới.
- Ưu tiên expand–migrate–contract.
- Backup và thử restore định kỳ.
- Không chạy migration nguy hiểm đồng thời trên nhiều replica.

13.4. Dockerfile .NET 10

```
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY ["Sales.Api/Sales.Api.csproj", "Sales.Api/"]
RUN dotnet restore "Sales.Api/Sales.Api.csproj"
COPY . .
WORKDIR "/src/Sales.Api"
RUN dotnet publish -c Release -o /app/publish /p:UseAppHost=false

FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS final
WORKDIR /app
COPY --from=build /app/publish .
USER $APP_UID
EXPOSE 8080
ENTRYPOINT ["dotnet", "Sales.Api.dll"]
```

Multi-stage build làm image runtime nhỏ hơn và không chứa SDK. Chạy non-root giảm mức ảnh hưởng khi container bị xâm nhập. Không copy secrets vào image.

13.5. CI cơ bản

```
name: api-ci
on:
  push:
    branches: [ main ]
  pull_request:
jobs:
```

```

build-test:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-dotnet@v4
  with:
    dotnet-version: '10.0.x'
    - run: dotnet restore
    - run: dotnet build --no-restore -c Release
    - run: dotnet test --no-build -c Release

```

13.6. Production readiness checklist

Nhóm	Tiêu chí
Cấu hình	Secrets ngoài source; ValidateOnStart; môi trường tách biệt.
Bảo mật	HTTPS, authorization tài nguyên, rate limit, scan dependency.
Dữ liệu	Migration versioned, backup/restore, index, transaction.
Tin cậy	Timeout, retry có kiểm soát, idempotency, health checks.
Quan sát	Structured logs, metrics, traces, alert.
Hiệu năng	Load test, giới hạn payload/pageSize, cache.
Triển khai	Image bất biến, non-root, rollback.
Tài liệu	OpenAPI, runbook, sơ đồ phụ thuộc.

PHỤ LỤC E. LỘ TRÌNH THỰC HÀNH 12 BUỔI

Buổi	Nội dung	Sản phẩm
1	HTTP, REST, middleware,	API mẫu + middleware.

	DI	
2	Binding, validation, OpenAPI	CRUD in-memory.
3	ADO.NET, Dapper	CRUD SQL Server.
4	Transaction, repository, paging	Order transaction.
5	JWT, refresh token	Auth module.
6	Security hardening	Resource auth + rate limit.
7	Problem Details, logging	Error contract + correlation.
8	Redis, resilience, health	Cache + health checks.
9	Clean Architecture, outbox	Orders module.
10	Testing	Unit + integration tests.
11	Docker, migration	Stack container.
12	CI/CD và bảo vệ	Pipeline + demo.

PHỤ LỤC F. CÂU HỎI ÔN TẬP NÂNG CAO

24. Vì sao 401 và 403 không thể dùng thay thế?
25. Khi nào trả 409 thay vì 400?
26. Vì sao POST thanh toán cần Idempotency-Key?
27. Phân biệt offset và keyset pagination.
28. Vì sao cache key phải chứa tenant?
29. Khi nào retry làm sự cố nặng hơn?
30. Vì sao không gọi dịch vụ ngoài trong SQL transaction?
31. Outbox giải quyết vấn đề nhất quán nào?
32. Resource authorization khác role authorization ra sao?
33. Vì sao Dapper cần integration test với DB thật?
34. Liveness khác readiness thế nào?
35. Điều gì tạo breaking change?
36. Vì sao không ghi token vào log?

37. Khi nào modular monolith tốt hơn microservices?

38. Expand–migrate–contract hỗ trợ zero downtime thế nào?

PHỤ LỤC G. CẤU TRÚC REPOSITORY ĐỀ XUẤT

```
Sales.sln
src/
  Sales.Api/
  Sales.Application/
  Sales.Domain/
  Sales.Infrastructure/
tests/
  Sales.Domain.Tests/
  Sales.Application.Tests/
  Sales.Api.IntegrationTests/
database/migrations/
deploy/
docs/openapi/
```

PHẦN BỔ SUNG NÂNG CAO III – CÁC CHỦ ĐỀ THEO YÊU CẦU THỰC TẾ DOANH NGHIỆP 2026

Phần này mở rộng thêm bốn bài học nâng cao mà các giáo trình Web API thường bỏ sót nhưng lại là yêu cầu phổ biến trong tuyển dụng và vận hành thực tế: truy cập dữ liệu với Entity Framework Core, giao tiếp bất đồng bộ/thời gian thực, khả năng quan sát và triển khai cloud-native, và các chủ đề bảo mật dữ liệu/vận hành nâng cao. Giảng viên có thể dùng làm module tự chọn (elective) hoặc bài đọc thêm cho sinh viên khá giỏi.

BÀI 14: ENTITY FRAMEWORK CORE CHUYÊN SÂU

14.1. Mục tiêu bài học

- Hiểu vai trò của DbContext, DbSet và Change Tracker.
- Ánh xạ Entity bằng Fluent API thay vì rải Data Annotation khắp Domain.
- Thực hiện migration an toàn trong vòng đời phát triển và production.

- Viết truy vấn hiệu năng cao: projection, AsNoTracking, Include có kiểm soát.
- Nhận diện và xử lý vấn đề N+1 query – lỗi hiệu năng phổ biến nhất khi dùng ORM.

14.2. DbContext, DbSet và Fluent API mapping

DbContext đại diện cho một phiên làm việc với cơ sở dữ liệu, gồm Change Tracker theo dõi trạng thái Entity (Added, Modified, Deleted, Unchanged). Nên cấu hình mapping bằng Fluent API trong IEntityTypeConfiguration để tách biệt Domain khỏi chi tiết lưu trữ.

```
public class ProductConfiguration : IEntityTypeConfiguration<Product>
{
    public void Configure(EntityTypeBuilder<Product> builder)
    {
        builder.ToTable("Products");
        builder.HasKey(p => p.Id);
        builder.Property(p => p.Name).HasMaxLength(200).IsRequired();
        builder.Property(p => p.Price).HasColumnType("decimal(18,2)");
        builder.HasIndex(p => p.Sku).IsUnique();
        builder.HasMany(p => p.OrderItems)
            .WithOne(oi => oi.Product)
            .HasForeignKey(oi => oi.ProductId)
            .OnDelete(DeleteBehavior.Restrict);
    }
}

public class SalesDbContext(DbContextOptions<SalesDbContext> options) :
    DbContext(options)
{
    public DbSet<Product> Products => Set<Product>();
    public DbSet<Order> Orders => Set<Order>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
```

```

{

modelBuilder.ApplyConfigurationsFromAssembly(typeof(SalesDbContext).Assembly);

}
}

```

Đăng ký DbContext trong Program.cs với vòng đời Scoped mặc định (đúng với vòng đời một request):

```

builder.Services.AddDbContext<SalesDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("Default"),
        sql => sql.EnableRetryOnFailure(3)));

```

14.3. Quản lý Migration

Lệnh	Mục đích
dotnet ef migrations add <Ten>	Tạo migration mới từ thay đổi Model
dotnet ef database update	Áp dụng migration vào CSDL đang cấu hình
dotnet ef migrations remove	Gỡ migration cuối cùng chưa apply
dotnet ef migrations script	Xuất script SQL để chạy thủ công/CI
dotnet ef database update <Ten>	Rollback về một migration cụ thể

Lưu ý: Không sửa trực tiếp một migration đã chạy ở production. Nếu cần đổi ý, tạo migration mới (cùng nguyên tắc expand–migrate–contract đã trình bày ở Bài 13). Trong CI/CD, ưu tiên chạy dotnet ef database update như một bước riêng, tách khỏi lần khởi động đầu tiên của ứng dụng.

14.4. Truy vấn nâng cao và tối ưu hiệu năng

- AsNoTracking(): dùng cho các truy vấn chỉ đọc để bỏ qua Change Tracker, giảm bộ nhớ và tăng tốc.
- Projection (Select trực tiếp sang DTO): chỉ lấy đúng cột cần thiết thay vì tải cả Entity.
- Include/ThenInclude có kiểm soát: chỉ nạp quan hệ thực sự cần dùng, tránh Cartesian explosion khi Include nhiều collection.

- Compiled Query (EF.CompileAsyncQuery): tăng tốc cho các truy vấn lặp lại rất nhiều lần với hình dạng cố định.

```
var result = await context.Orders
    .Where(o => o.CustomerId == customerId)
    .OrderByDescending(o => o.CreatedAt)
    .Select(o => new OrderSummaryDto(o.Id, o.Status, o.Total, o.CreatedAt))
    .AsNoTracking()
    .ToListAsync(cancellationToken);
```

14.4.1. Vấn đề N+1 Query

N+1 xảy ra khi ứng dụng chạy 1 truy vấn lấy danh sách cha, rồi lặp qua từng bản ghi để chạy thêm 1 truy vấn con — tổng cộng N+1 lần round-trip tới CSDL thay vì 1 hoặc 2 lần.

```
// Sai: N+1 – mỗi Order kích hoạt thêm 1 query lazy-load Customer
var orders = await context.Orders.ToListAsync();
foreach (var o in orders)
    Console.WriteLine(o.Customer.FullName); // truy vấn ngầm mỗi vòng lặp

// Đúng: gộp lại bằng Include hoặc projection
var orders = await context.Orders
    .Include(o => o.Customer)
    .AsNoTracking()
    .ToListAsync();
```

Lưu ý: Bật ghi log câu lệnh SQL (LogTo trong DbContextOptions) khi phát triển để phát hiện sớm các truy vấn lặp lại bất thường. Công cụ như MiniProfiler hoặc EF Core logging category `Microsoft.EntityFrameworkCore.Database.Command` giúp quan sát số lượng round-trip thực tế.

14.5. So sánh sâu Dapper và EF Core: khi nào dùng cái nào

Tiêu chí	Dapper	EF Core
Hiệu năng truy vấn	Kiểm soát SQL tuyệt đối, thường nhanh	Nhanh với truy vấn đơn

phức tạp	hơn	giản; cần review SQL sinh ra
Tốc độ phát triển CRUD	Phải tự viết nhiều SQL lặp lại	Sinh migration, LINQ ngắn gọn
Độ khó học	Cần vững SQL	Cần hiểu Change Tracker, migration
Phù hợp	Báo cáo, truy vấn phức tạp, hệ thống hiệu năng cao	CRUD nghiệp vụ, đội cần tốc độ phát triển
Kết hợp thực tế	Dùng chung DbContext.Database.GetDbConnection() để tận dụng cả hai	

Nhiều hệ thống thực tế dùng kết hợp: EF Core cho các thao tác CRUD/ng nghiệp vụ thông thường, Dapper cho báo cáo, dashboard hoặc truy vấn tổng hợp phức tạp cần kiểm soát chỉ số/kế hoạch thực thi.

14.6. Bài tập

39. Chuyển một Repository Dapper (Bài 3) sang EF Core, giữ nguyên interface.
40. Viết truy vấn Order kèm OrderItems bằng Include, đo số lượng câu SQL sinh ra và tối ưu về projection.
41. Tạo migration thêm cột DiscountPercent cho Product và xuất migration script.
42. Dựng một ví dụ N+1 thật trên dữ liệu mẫu, chụp lại log SQL trước/sau khi sửa.

BÀI 15: GIAO TIẾP THỜI GIAN THỰC VÀ XỬ LÝ BẤT ĐỒNG BỘ

15.1. Mục tiêu bài học

- Phân biệt các mô hình giao tiếp: request/response, real-time hai chiều, streaming một chiều, message queue bất đồng bộ.
- Xây dựng kênh real-time cơ bản với SignalR.
- Triển khai tác vụ nền chuẩn với IHostedService/BackgroundService.
- Hiểu vai trò của Message Queue (RabbitMQ) trong kiến trúc hướng sự kiện.

15.2. SignalR – giao tiếp hai chiều thời gian thực

SignalR phù hợp cho thông báo tức thời, cập nhật trạng thái đơn hàng, chat, dashboard trực tiếp. SignalR tự chọn giao thức vận chuyển tốt nhất (WebSocket, Server-Sent Events, Long Polling) tùy trình duyệt/hạ tầng.

```
public class OrderNotificationHub : Hub
{
    public async Task JoinCustomerGroup(string customerId)
        => await Groups.AddToGroupAsync(Context.ConnectionId, $"customer-
{customerId}");
}

// Program.cs
builder.Services.AddSignalR();
app.MapHub<OrderNotificationHub>("/hubs/orders");

// Gửi thông báo từ OrderService khi trạng thái đơn hàng thay đổi
await hubContext.Clients.Group($"customer-{order.CustomerId}")
    .SendAsync("OrderStatusChanged", new { order.Id, order.Status });
```

Lưu ý: SignalR cần cấu hình sticky session hoặc Redis backplane khi chạy nhiều instance (scale-out), vì kết nối WebSocket gắn với một instance cụ thể.

15.3. Server-Sent Events (SSE) cho luồng dữ liệu một chiều

Khi chỉ cần server đẩy dữ liệu một chiều xuống client (ví dụ luồng tiến độ xử lý, log trực tiếp), SSE đơn giản hơn SignalR vì chạy trên HTTP thuần, không cần thư viện client riêng.

```
app.MapGet("/api/orders/{id}/progress-stream", async (int id, HttpContext ctx) =>
{
    ctx.Response.Headers.ContentType = "text/event-stream";
    for (var i = 0; i <= 100; i += 20)
    {
```

```

        await ctx.Response.WriteAsync($"data: {i}%\n\n");
        await ctx.Response.Body.FlushAsync();
        await Task.Delay(500);
    }
});

```

15.4. Background Services chi tiết

BackgroundService là lớp trừu tượng triển khai IHostedService, phù hợp cho tác vụ chạy nền dài hạn: xử lý hàng đợi, dọn dữ liệu định kỳ, gửi email hàng loạt.

```

public class OrderReminderWorker(
    IServiceScopeFactory scopeFactory,
    ILogger<OrderReminderWorker> logger) : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        using var timer = new PeriodicTimer(TimeSpan.FromMinutes(15));
        while (await timer.WaitForNextTickAsync(stoppingToken))
        {
            using var scope = scopeFactory.CreateScope();
            var orderService = scope.ServiceProvider.GetRequiredService<IOrderService>();
            await orderService.SendPendingReminders(stoppingToken);
        }
    }
}

// Program.cs
builder.Services.AddHostedService<OrderReminderWorker>();

```

Lưu ý: BackgroundService chạy trong cùng tiến trình với API nên chia sẻ tài nguyên (CPU, memory) với việc phục vụ request. Tác vụ nặng hoặc cần scale độc lập nên tách thành Worker Service/consumer riêng.

15.5. Kiến trúc hướng sự kiện với Message Queue (RabbitMQ)

Message Queue giúp tách rời (decouple) các service: service phát sự kiện (publisher) không cần biết ai đang lắng nghe (consumer), tăng khả năng chịu lỗi và mở rộng độc lập.

```
// Publisher – sau khi tạo đơn hàng thành công
var message = JsonSerializer.SerializeToUtf8Bytes(new OrderCreatedEvent(order.Id,
order.CustomerId, order.Total));
channel.BasicPublish(exchange: "orders", routingKey: "order.created", body: message);

// Consumer – service Inventory lắng nghe để trừ tồn kho
var consumer = new EventingBasicConsumer(channel);
consumer.Received += async (_, ea) =>
{
    var evt = JsonSerializer.Deserialize<OrderCreatedEvent>(ea.Body.Span);
    await inventoryService.ReserveStock(evt!.OrderId);
    channel.BasicAck(ea.DeliveryTag, multiple: false);
};
```

- Kết hợp Outbox Pattern (Bài 12) với Message Queue để đảm bảo sự kiện không bị mất khi service crash giữa lúc ghi CSDL và publish message.
- Idempotent consumer: consumer phải xử lý an toàn khi nhận trùng message (do retry, redelivery).
- Dead-letter queue: hàng đợi phụ chứa message xử lý thất bại nhiều lần, phục vụ điều tra thay vì mất dữ liệu.

15.6. Bài tập

43. Xây dựng SignalR Hub thông báo khi đơn hàng chuyển sang trạng thái Shipped.
44. Viết BackgroundService quét đơn hàng quá hạn thanh toán mỗi 10 phút và tự hủy.
45. Thiết kế sự kiện OrderCreatedEvent và mô tả luồng publish/consume giữa Order Service và Inventory Service.

BÀI 16: QUAN SÁT HỆ THỐNG NÂNG CAO VÀ VẬN HÀNH CLOUD-NATIVE

16.1. Mục tiêu bài học

- Hiểu ba trụ cột Observability: Logs, Metrics, Traces và cách OpenTelemetry hợp nhất chúng.
- Theo dõi một request xuyên nhiều service bằng Correlation ID/Trace ID.
- Đóng gói và mô tả triển khai cơ bản trên Kubernetes.
- Hiểu khái niệm Feature Flag, triển khai Blue-Green/Canary và kiểm thử tải.

16.2. OpenTelemetry: hợp nhất Traces, Metrics, Logs

OpenTelemetry (OTel) là chuẩn mở, độc lập nhà cung cấp, cho phép thu thập traces/metrics/logs và xuất tới nhiều backend (Grafana, Jaeger, Azure Monitor, Datadog...) mà không khoá chặt vào một hệ thống cụ thể.

```
builder.Services.AddOpenTelemetry()
    .ConfigureResource(r => r.AddService("Sales.Api"))
    .WithTracing(tracing => tracing
        .AddAspNetCoreInstrumentation()
        .AddHttpClientInstrumentation()
        .AddSqlClientInstrumentation()
        .AddOtlpExporter())
    .WithMetrics(metrics => metrics
        .AddAspNetCoreInstrumentation()
        .AddRuntimeInstrumentation()
        .AddOtlpExporter());
```

16.3. Correlation ID và Distributed Tracing xuyên service

Mỗi request nên mang một Trace ID xuyên suốt từ Gateway tới các service phía sau, để có thể ghép lại toàn bộ hành trình xử lý khi tra cứu log hoặc điều tra sự cố.

```
app.Use(async (context, next) =>
{
    var traceId = context.Request.Headers["X-Trace-Id"].FirstOrDefault()
        ?? Guid.NewGuid().ToString("N");
```

```

context.Response.Headers["X-Trace-Id"] = traceId;
using (Serilog.Context.LogContext.PushProperty("TraceId", traceId))
    await next();
});

```

Lưu ý: Khi gọi service khác qua HttpClient, phải truyền tiếp header X-Trace-Id (hoặc chuẩn W3C traceparent mà OpenTelemetry dùng mặc định) để không đứt gãy chuỗi trace giữa các service.

16.4. Đóng gói và triển khai Kubernetes cơ bản

Kubernetes (K8s) là nền tảng điều phối container phổ biến nhất cho production. Bốn tài nguyên cơ bản cần nắm:

Tài nguyên	Vai trò
Deployment	Quản lý số lượng bản sao (replica) của Pod, tự khởi động lại khi lỗi
Service	Địa chỉ mạng ổn định, cân bằng tải giữa các Pod
ConfigMap	Cấu hình không nhạy cảm (appsettings theo môi trường)
Secret	Chuỗi kết nối, khoá JWT, mật khẩu – mã hoá base64, tích hợp Vault khi cần

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: sales-api
spec:
  replicas: 3
  selector:
    matchLabels: { app: sales-api }

```

```

template:
  metadata:
    labels: { app: sales-api }
  spec:
    containers:
      - name: sales-api
        image: registry.example.com/sales-api:1.4.0
        ports: [{ containerPort: 8080 }]
        envFrom:
          - configMapRef: { name: sales-api-config }
          - secretRef: { name: sales-api-secrets }
        readinessProbe:
          httpGet: { path: /health/ready, port: 8080 }
        livenessProbe:
          httpGet: { path: /health/live, port: 8080 }
---
apiVersion: v1
kind: Service
metadata: { name: sales-api }
spec:
  selector: { app: sales-api }
  ports: [{ port: 80, targetPort: 8080 }]

```

- HorizontalPodAutoscaler (HPA): tự động tăng/giảm số Pod dựa trên CPU/metric tùy chỉnh.
- readinessProbe/livenessProbe nên trở tới đúng hai endpoint health checks đã xây dựng ở Bài 5/11 (/health/ready, /health/live).

16.5. Feature Flag và triển khai an toàn

Chiến lược	Mô tả	Rủi ro
Feature Flag	Bật/tắt tính năng bằng cấu hình, không cần deploy lại	Nợ kỹ thuật nếu không dọn flag cũ

Blue-Green	Chạy song song 2 môi trường, chuyển traffic tức thời	Cần gấp đôi tài nguyên tạm thời
Canary	Đưa traffic dần dần (5%→25%→100%) sang bản mới	Cần giám sát chặt theo thời gian thực

16.6. Kiểm thử tải với k6

Kiểm thử tải giúp xác định điểm giới hạn của hệ thống trước khi production gặp sự cố thực. k6 là công cụ mã nguồn mở, viết kịch bản bằng JavaScript, dễ tích hợp CI.

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  vus: 50,
  duration: '30s',
};

export default function () {
  const res = http.get('https://api.example.com/api/products?page=1&pageSize=20');
  check(res, { 'status 200': (r) => r.status === 200 });
  sleep(1);
}
```

16.7. Bài tập

46. Thêm OpenTelemetry tracing cho Sales.Api và xuất trace ra console exporter.
47. Viết Deployment + Service YAML cho một service bất kỳ trong đồ án, kèm readiness/liveness probe.
48. Viết kịch bản k6 kiểm thử tải endpoint đăng nhập với 30 người dùng ảo trong 20 giây.

BÀI 17: BẢO VỆ DỮ LIỆU, TUÂN THỦ VÀ CÁC CHỦ ĐỀ VẬN HÀNH NÂNG CAO

17.1. Mục tiêu bài học

- Áp dụng nguyên tắc bảo vệ dữ liệu cá nhân (tối thiểu hoá dữ liệu, ẩn danh hoá) vào thiết kế API.
- Triển khai audit logging và soft delete đúng cách.
- Thiết kế webhook an toàn cho cả vai trò nhà cung cấp và bên nhận.
- Sinh tài liệu API hiện đại và xử lý upload file lớn.

17.2. Bảo vệ dữ liệu cá nhân trong thiết kế API

- Data minimization: chỉ trả về trường dữ liệu thực sự cần thiết cho từng use case, không trả nguyên Entity.
- Ẩn danh hoá/che dữ liệu nhạy cảm trong log (không log số thẻ, mật khẩu, token đầy đủ).
- Cung cấp endpoint cho quyền của chủ thể dữ liệu: xem, xuất, yêu cầu xoá dữ liệu cá nhân.
- Mã hoá dữ liệu nhạy cảm khi lưu trữ (at rest) và khi truyền (in transit qua HTTPS).

Lưu ý: Đây là nguyên tắc kỹ thuật mang tính tham khảo giáo dục, không thay thế tư vấn pháp lý. Hệ thống xử lý dữ liệu cá nhân thực tế cần đối chiếu quy định pháp luật hiện hành tại khu vực triển khai.

17.3. Audit Logging và Soft Delete

Audit log ghi lại ai đã làm gì, khi nào, với dữ liệu nào — phục vụ điều tra sự cố và tuân thủ. Soft delete đánh dấu bản ghi đã xoá thay vì xoá vật lý, giữ lại lịch sử.

```
public class AuditEntry
{
    public long Id { get; set; }
    public string EntityName { get; set; } = default!;
    public string Action { get; set; } = default!; // Created, Updated, Deleted
    public string UserId { get; set; } = default!;
    public DateTime OccurredAtUtc { get; set; }
    public string? Changes { get; set; } // JSON diff
}
```

```
// Soft delete: thêm cột IsDeleted, lọc mặc định bằng Global Query Filter
modelBuilder.Entity<Product>().HasQueryFilter(p => !p.IsDeleted);
```

Lưu ý: Global Query Filter tự động loại các bản ghi đã soft-delete khỏi mọi truy vấn, tránh việc quên thêm điều kiện IsDeleted == false ở từng nơi.

17.4. Thiết kế Webhook an toàn

Webhook cho phép hệ thống chủ động gọi ngược (push) sự kiện tới URL của bên thứ ba, thay vì để họ liên tục polling.

- Bên gửi: ký payload bằng HMAC (secret riêng cho từng subscriber) để bên nhận xác minh nguồn gốc.
- Bên gửi: cần retry có backoff khi endpoint nhận lỗi, và giới hạn số lần retry.
- Bên nhận: luôn xác minh chữ ký HMAC trước khi xử lý, kiểm tra timestamp để chống replay attack.

```
// Bên gửi – ký payload
var payload = JsonSerializer.Serialize(webhookEvent);
var signature = Convert.ToHexString(
    HMACSHA256.HashData(Encoding.UTF8.GetBytes(secret),
    Encoding.UTF8.GetBytes(payload)));
request.Headers.Add("X-Signature", signature);
```

17.5. Tài liệu API nâng cao (Scalar/Redoc)

Ngoài Swagger UI (Bài 2), các công cụ như Scalar hoặc Redoc render tài liệu OpenAPI đẹp và thân thiện hơn cho người dùng ngoài, thường dùng cho cổng API công khai (developer portal).

```
// .NET 10 hỗ trợ sẵn tài liệu OpenAPI; thêm Scalar UI:
app.MapOpenApi();
app.MapScalarApiReference(options =>
{
    options.Title = "Sales API Reference";
});
```

17.6. Upload file lớn và xử lý bất đồng bộ

- Không đọc toàn bộ file vào bộ nhớ; dùng Stream và giới hạn kích thước qua `RequestSizeLimit`.
- Với file rất lớn, dùng chunked upload: client chia nhỏ, server ghép lại theo thứ tự.
- Xử lý file sau khi nhận (ví dụ resize ảnh, quét virus) nên đưa vào hàng đợi/`BackgroundService` thay vì xử lý đồng bộ trong request.

```
[RequestSizeLimit(50_000_000)]
[HttpPost("products/{id}/image")]
public async Task<IActionResult> UploadImage(int id, IFormFile file, CancellationToken
ct)
{
    if (file.Length == 0) return BadRequest("File rỗng.");
    await using var stream = file.OpenReadStream();
    var storedPath = await _storage.SaveAsync(stream, file.FileName, ct);
    await _queue.EnqueueImageProcessing(id, storedPath, ct);
    return Accepted(new { storedPath });
}
```

17.7. Bài tập

49. Thêm cột `IsDeleted` và Global Query Filter cho Entity `Product`; kiểm tra bản ghi soft-delete không còn xuất hiện trong GET.
50. Thiết kế bảng `AuditEntry` và ghi log khi đơn hàng đổi trạng thái.
51. Viết endpoint gửi webhook có ký HMAC và một endpoint mẫu xác minh chữ ký ở phía nhận.
52. Xây dựng endpoint upload ảnh sản phẩm giới hạn 5MB, trả 202 Accepted và xử lý resize ở `BackgroundService`.

PHỤ LỤC H. BẢNG TỔNG HỢP CÔNG CỤ VÀ THƯ VIỆN MỞ RỘNG

Nhóm	Công cụ/Thư viện	Vai trò
Truy cập dữ liệu	EF Core, Dapper	ORM đầy đủ tính năng / Micro-ORM hiệu năng cao

Real-time	SignalR, Server-Sent Events	Đẩy dữ liệu thời gian thực tới client
Message Queue	RabbitMQ, Azure Service Bus, Kafka	Giao tiếp bất đồng bộ, kiến trúc hướng sự kiện
Observability	OpenTelemetry, Grafana, Jaeger, Seq	Thu thập và trực quan hoá logs/metrics/traces
Triển khai	Docker, Kubernetes, Helm	Đóng gói và điều phối container
Kiểm thử tải	k6, JMeter	Đo hiệu năng và điểm giới hạn hệ thống
Tài liệu API	Swagger UI, Scalar, Redoc	Hiển thị tài liệu OpenAPI cho nhà phát triển
Bảo mật bổ sung	HMAC, Vault/Key Vault, OWASP ZAP	Ký dữ liệu, quản lý secrets, quét lỗ hổng

PHỤ LỤC I. GỢI Ý ĐÁP ÁN NGẮN CHO CÂU HỎI ÔN TẬP NÂNG CAO (PHỤ LỤC F)

Đây là gợi ý ý chính, không phải đáp án đầy đủ; giảng viên/học viên nên diễn giải và bổ sung ví dụ minh hoạ.

53. 401 báo chưa xác thực (thiếu/sai token); 403 báo đã xác thực nhưng không đủ quyền — trả sai mã sẽ khiến client xử lý luồng đăng nhập lại không chính xác.
54. Trả 409 khi request hợp lệ về cú pháp nhưng xung đột với trạng thái hiện tại của tài nguyên (ví dụ trùng SKU); 400 dành cho lỗi cú pháp/dữ liệu đầu vào không hợp lệ.
55. Idempotency-Key giúp client gọi lại an toàn khi mất kết nối giữa lúc thanh toán, tránh tạo giao dịch trùng.
56. Offset pagination đơn giản nhưng chậm/lệch dữ liệu khi tập dữ liệu thay đổi liên tục; keyset pagination ổn định và nhanh hơn với tập dữ liệu lớn.
57. Cache key phải chứa tenant để tránh rò rỉ dữ liệu giữa các khách hàng trong hệ thống multi-tenant.

58. Retry có thể làm nặng thêm sự cố khi hệ thống đích đang quá tải (retry storm) — cần backoff và giới hạn số lần thử.
59. Không gọi dịch vụ ngoài trong transaction SQL vì transaction sẽ giữ khoá lâu, tăng nguy cơ deadlock và timeout.
60. Outbox đảm bảo việc ghi dữ liệu và phát sự kiện xảy ra nhất quán (cùng transaction cục bộ), tránh mất sự kiện khi service crash.
61. Resource authorization kiểm tra quyền trên chính đối tượng dữ liệu (ví dụ chỉ chủ đơn hàng mới xem được); role authorization chỉ kiểm tra vai trò chung chung.
62. Dapper không có Change Tracker/migration nên cần integration test với DB thật để phát hiện lỗi SQL/mapping không lộ ra lúc biên dịch.
63. Liveness kiểm tra tiến trình còn sống hay cần khởi động lại; readiness kiểm tra đã sẵn sàng nhận traffic (phụ thuộc ngoài đã kết nối được) hay chưa.
64. Breaking change là thay đổi khiến client hiện tại ngừng hoạt động: đổi kiểu dữ liệu, xoá trường, đổi ý nghĩa mã lỗi, đổi route.
65. Token trong log có thể bị đánh cắp nếu log rò rỉ hoặc bị truy cập trái phép — luôn mask/loại bỏ token khỏi log.
66. Modular monolith phù hợp khi đội ngũ nhỏ, nghiệp vụ chưa ổn định, muốn tốc độ phát triển nhanh và tránh chi phí vận hành phân tán quá sớm.
67. Expand–migrate–contract chia thay đổi schema thành nhiều bước nhỏ tương thích ngược, cho phép deploy tuần tự nhiều service mà không cần dừng hệ thống.

TÀI LIỆU THAM KHẢO BỔ SUNG – NGUỒN CHÍNH THỨC

- Microsoft Learn – ASP.NET Core documentation, phiên bản 10.0.
- Microsoft Learn – ASP.NET Core fundamentals và OpenAPI support.
- Microsoft Learn – Error handling và IExceptionHandler.
- Microsoft Learn – JWT bearer authentication và authorization.
- Microsoft Learn – Rate limiting middleware.
- Microsoft Learn – Docker containers for ASP.NET Core.
- IETF RFC 9110 – HTTP Semantics.

- IETF RFC 9457 – Problem Details for HTTP APIs.
- OpenAPI Specification.
- OWASP API Security Top 10.